Tema 4: Circuitos combinacionales II.

4.1: Introducción a los bloques funcionales combinacionales (comerciales).

4.2: Circuitos combinacionales MSI.

- 4.2.1: Decodificadores y demultiplexores.
- 4.2.2: Codificadores.
- 4.2.3: Multiplexores.
- 4.2.4: Comparadores.
- 4.2.5: Generadores / detectores de paridad.
- 4.2.6: Convertidores de código.
- 4.2.7: Circuitos aritméticos.

4.3: Análisis y síntesis de circuitos combinacionales utilizando circuitos integrados SSI y MSI.

"La libertad es el derecho a decirle a la gente lo que no quieren escuchar"

"If you hate violence and don't believe in politics, the only major remedy remaining is education".

George Orwell

4.1: Introducción.

- Escala SSI (hasta 10 puertas/chip) ⇔ funciones sencillas (pocas).

- Escala MSI (hasta 100 puertas/chip) ⇒ realización de una o varias funciones (complejas) de aplicación general.

- Ventajas de la utilización de ICs MSI:

_ Se reduce el número de ICs a utilizar.

_ Se reduce el coste del circuito.

_ Se simplifica el diseño (teórico).

_ Se simplifica el cableado del circuito y su construcción.

_ Lograr que el circuito no presente fenómenos aleatorios y problemas de EMC (*compatibilidad electromagnética*) resulta más fácil.

- La filosofía de diseño cambió con la aparición de los ICs de la escala MSI:

  _ Objetivo *principal*: minimizar el número de ICs a utilizar.

  _ Objetivo *secundario*: minimizar el número de puertas lógicas a utilizar.

- En este tema se estudian los principales bloques funcionales combinacionales pertenecientes a la escala de integración media (MSI). Dichos bloques *se utilizan habitualmente en la descripción de la arquitectura interna de circuitos digitales complejos como son, por ejemplo, procesadores, microcontroladores, DSPs-DSCs, etc*.

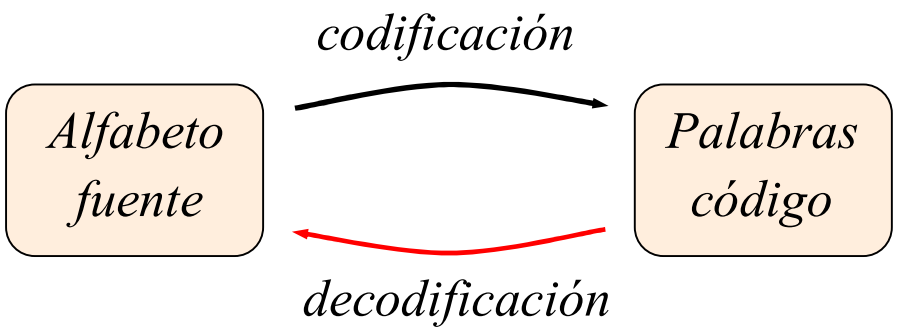
Comentario: los circuitos digitales no se diseñan siguiendo la filosofía que aquí se explica desde finales de los años 80… la existencia de los lenguajes de descripción de hardware, de las FPGAs y de las CPLDs ha cambiado el procedimiento de diseño e implementación de los circuitos digitales. Ahora bien, el conocimiento de los bloques funcionales que se estudian en este tema y, en general, en esta asignatura son esenciales para poder diseñar circuitos digitales y para poder entender la arquitectura interna de los procesadores, µC, DSPs y de los ASICs digitales.

- Decodificadores de n líneas de entrada a m líneas de salida

  de 1 línea de salida activa entre m líneas de salida

DEF. 1: Un decodificador es un circuito combinacional que decodifica información codificada en un código dado…. decodifica *palabras código*… indica el símbolo correspondiente a la palabra código presente en sus entradas.

Un decodificador X/Y indica para cada *palabra código* de un código X el *símbolo* del código fuente Y que le corresponde.



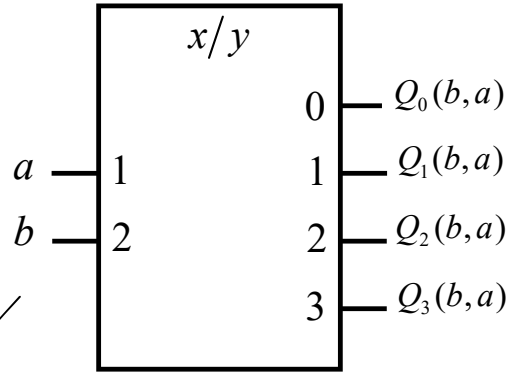
Ejemplo:

Alfabeto Fuente	Palabras código $BCD_{natural}$	Alfabeto código
0	0 0 0 0	0
1	0 0 0 1	1
2	0 0 1 0	
3	0 0 1 1	
4	0 1 0 0	
5	0 1 0 1	
6	0 1 1 0	
7	0 1 1 1	
8	1 0 0 0	
9	1 0 0 1	

DEF. 2: Un decodificador es un circuito combinacional con *n* entradas y *m* salidas. Para cada combinación (válida) de las *n* variables de entrada sólo se activa una de las *m* salidas. Dicho de otro modo, *permite seleccionar la salida que está activa*.

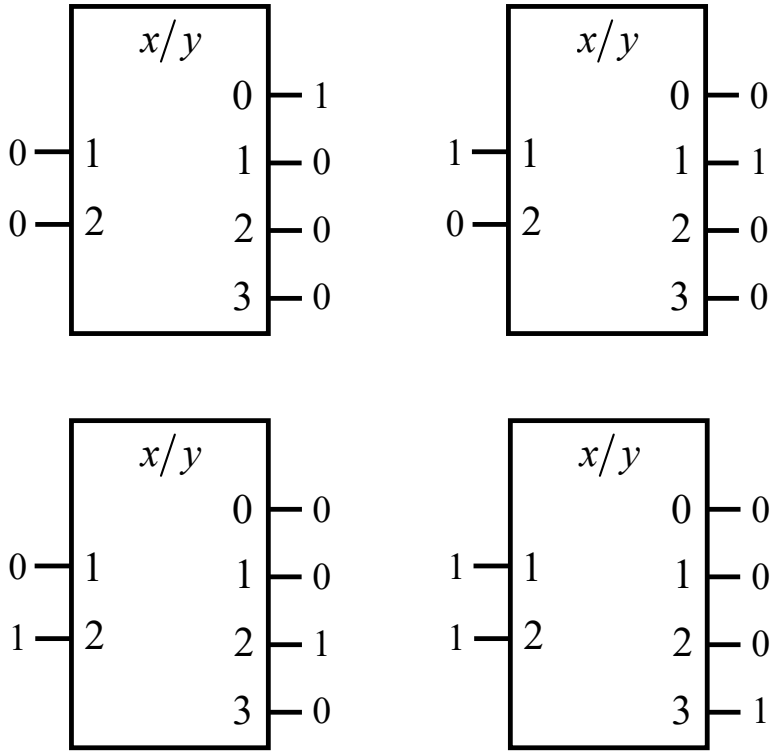
Nota: todos los decodificadores que se estudian en esta asignatura cumplen la definición anterior. Ahora bien, existen decodificadores, que no se estudian en esta asignatura por falta de tiempo, que no cumplen la definición anterior como, por ejemplo, un decodificador de *BCD* a 7 *segmentos*.

Decodificador de 2 a 4 ó de 1 entre 4

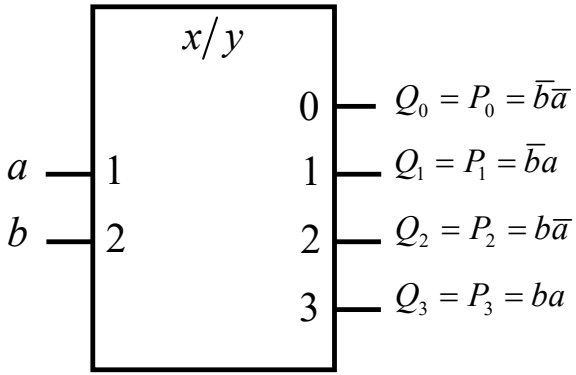


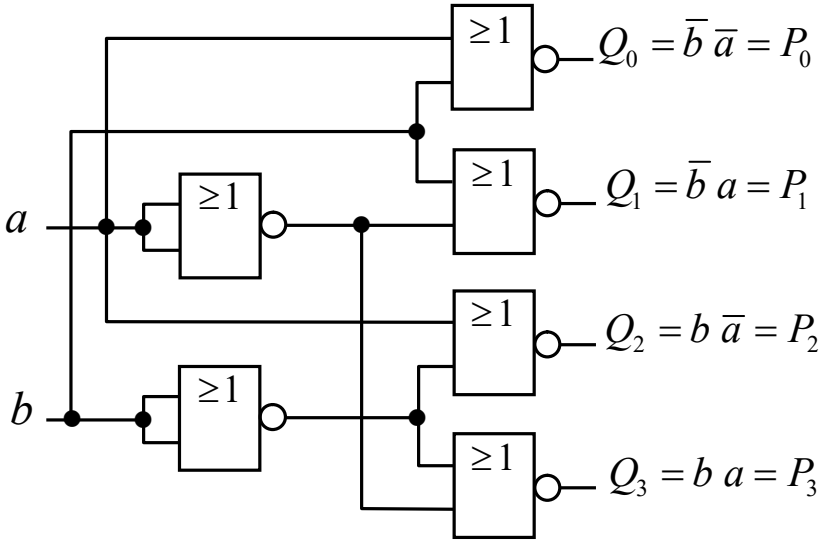
2	1				
b	a	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Las entradas tienen asignados pesos (la entrada *b* tiene asignado el peso 2 y la entrada *a* tiene asignado el peso 1). Las salidas están numeradas empezando desde 0. Se pone a 1 la salida que tiene asignado un número igual a la suma de los pesos de las entradas que están a 1.

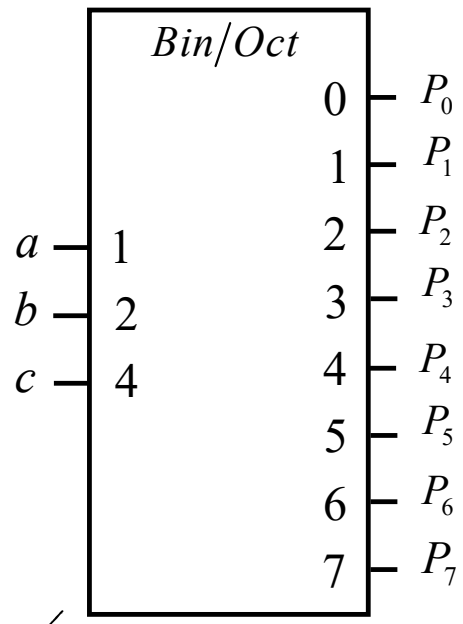


Implementación de un decodificador de 2 *a* 4 utilizando puertas NOR de 2 entradas





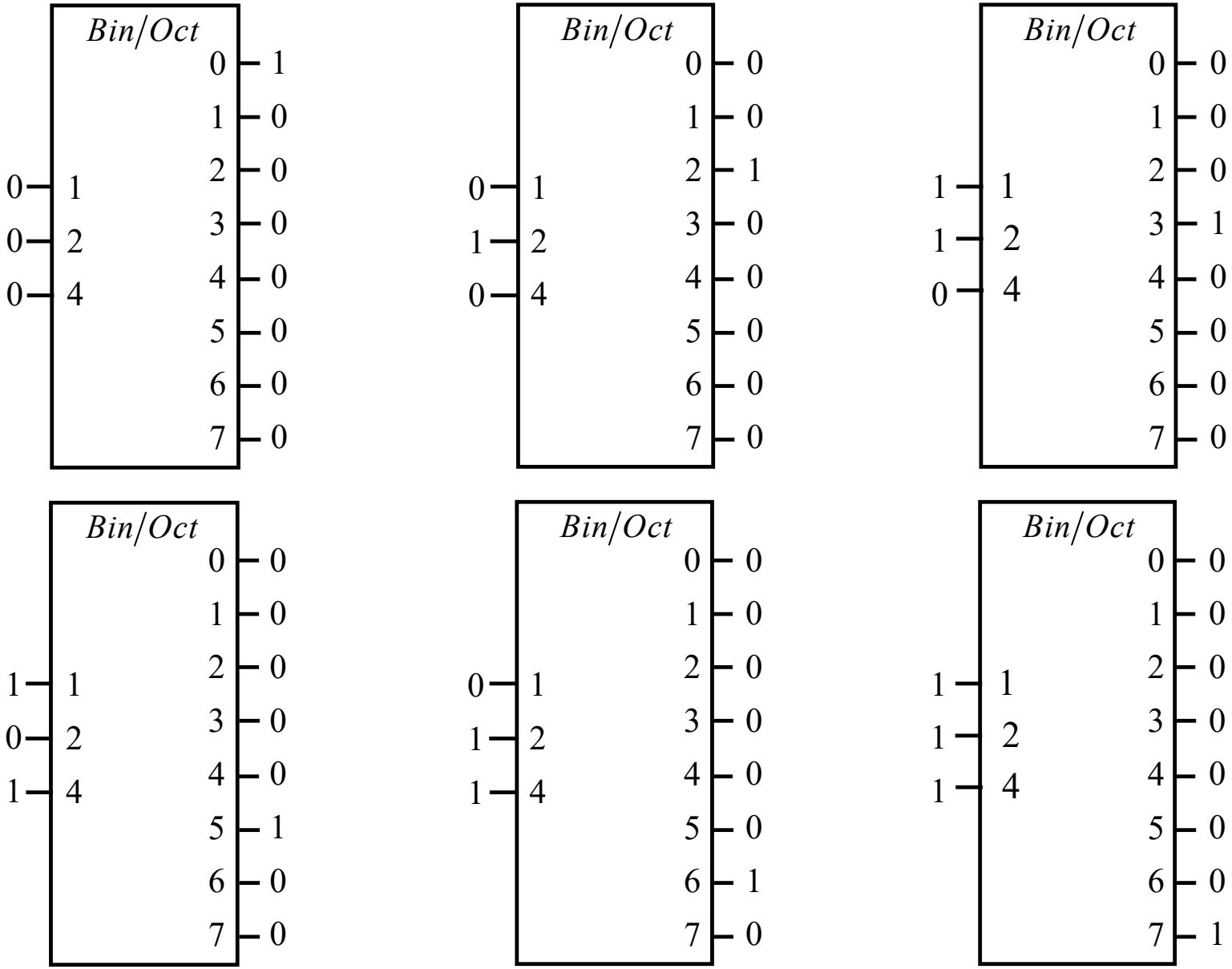
Decodificador de 3 a 8 o de 1 entre 8 ≡ Decodificador octal



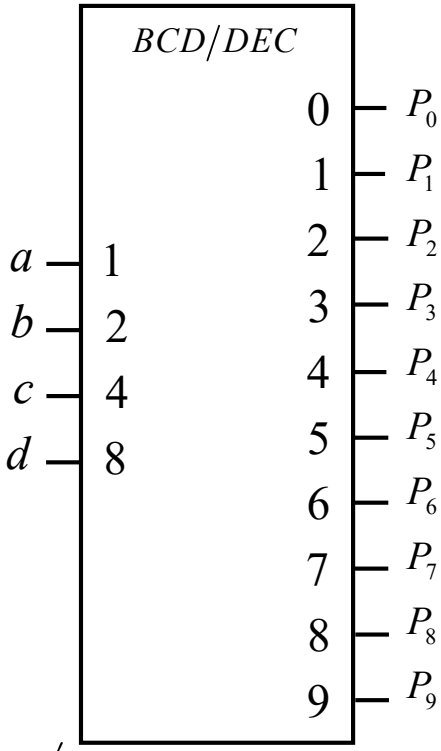
4	2	1								
c	b	a	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

Se pone a 1 la salida que tiene asignado un número igual a la suma de los pesos de las entradas que están a 1. Los pesos asociados a las entradas *c*, *b* y *a* son 4, 2 y 1 respectivamente.

Ejemplos:


Bin/Oct
0 1 0
0 2 0
0 4 0
0 1
1 2
2 3
3 4
4 5
5 6
6 7
7 0
Bin/Oct
0 1 0
1 2 0
0 4 1
0 1
1 2
2 3
3 4
4 5
5 6
6 7
7 0
Bin/Oct
1 1 0
1 2 0
0 4 1
0 1
1 2
2 3
3 4
4 5
5 6
6 7
7 0
Bin/Oct
1 1 0
0 2 0
1 4 1
0 1
1 2
2 3
3 4
4 5
5 1
6 6
7 7
0
Bin/Oct
0 1 0
1 2 0
1 4 1
0 1
1 2
2 3
3 4
4 5
5 6
6 1
7 7
0
Bin/Oct
1 1 0
1 2 0
1 4 1
0 1
1 2
2 3
3 4
4 5
5 6
6 7
7 1

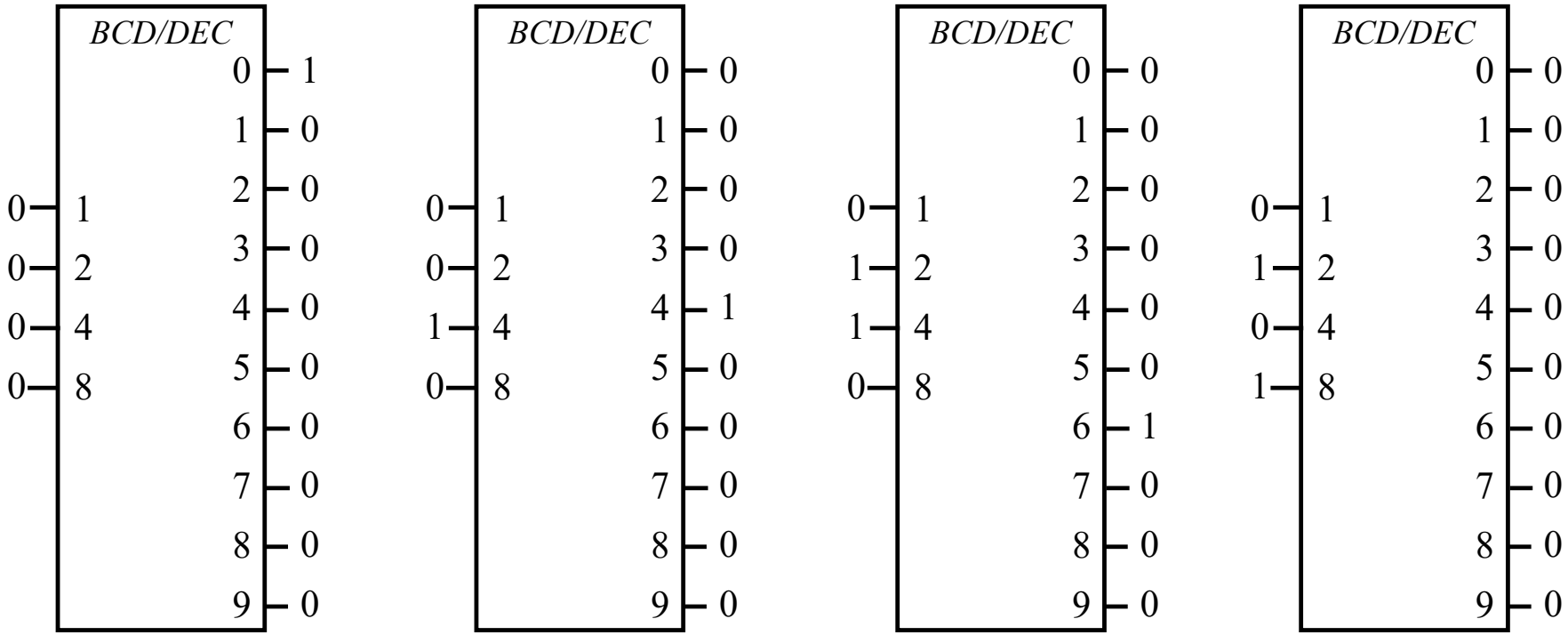
Decodificador decimal



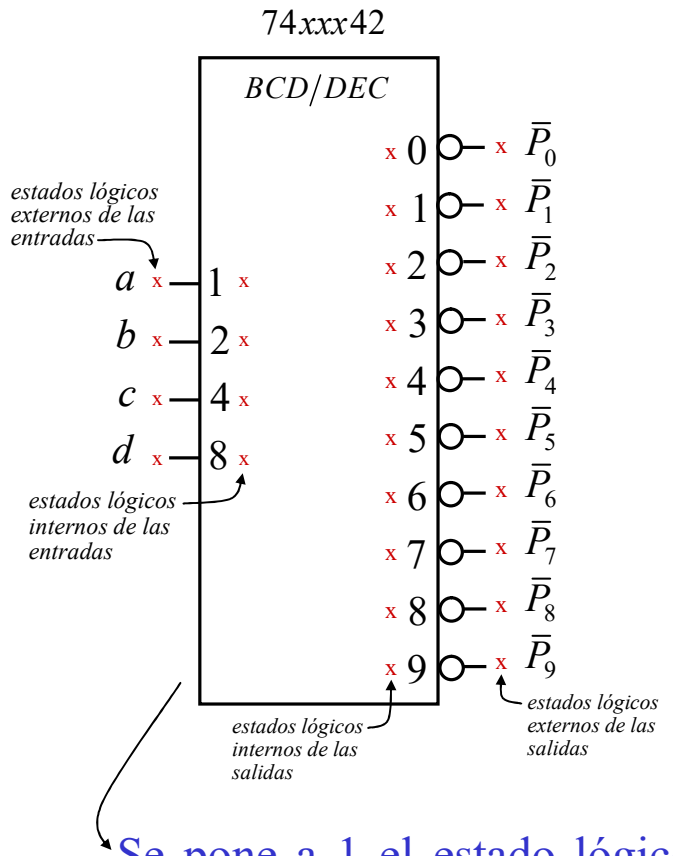
8	4	2	1										
d	c	b	a	P_9	P_8	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0
0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0	0	0	0

Se pone a 1 la salida que tiene asignado un número igual a la suma de los pesos de las entradas que están a 1. Los pesos asociados a las entradas *d*, *c*, *b* y *a* son 8, 4, 2 y 1 respectivamente. El mayor valor que se puede decodificar es el 1001_2

Ejemplos:


BCD/DEC
0 1 0 0
1 2 4 8
0 1 2 3 4 5 6 7 8 9
1 0 0 0 0 0 0 0 0 0
BCD/DEC
0 0 1 0
1 2 4 8
0 1 2 3 4 5 6 7 8 9
0 0 0 0 1 0 0 0 0 0
BCD/DEC
0 1 1 0
1 2 4 8
0 1 2 3 4 5 6 7 8 9
0 0 0 0 0 0 1 0 0 0
BCD/DEC
0 1 0 1
1 2 4 8
0 1 2 3 4 5 6 7 8 9
0 0 0 0 0 0 0 0 0 0

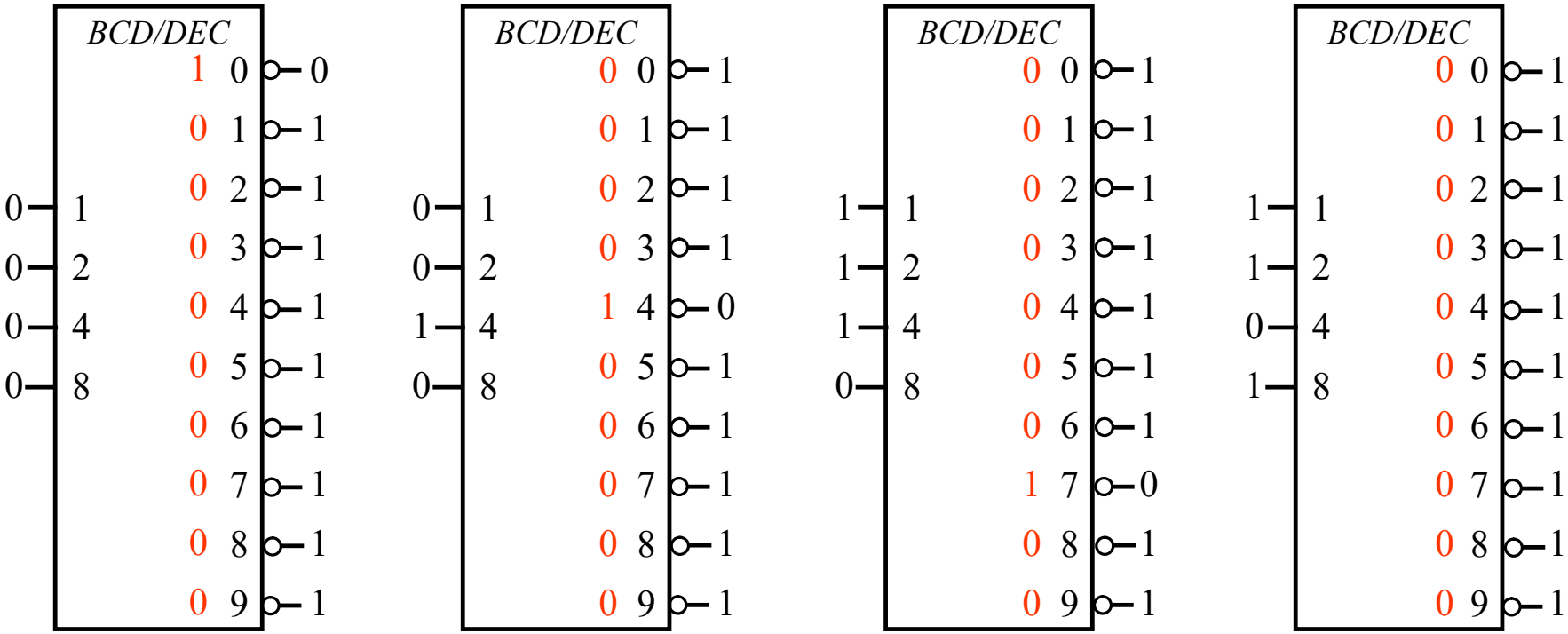
Decodificador decimal



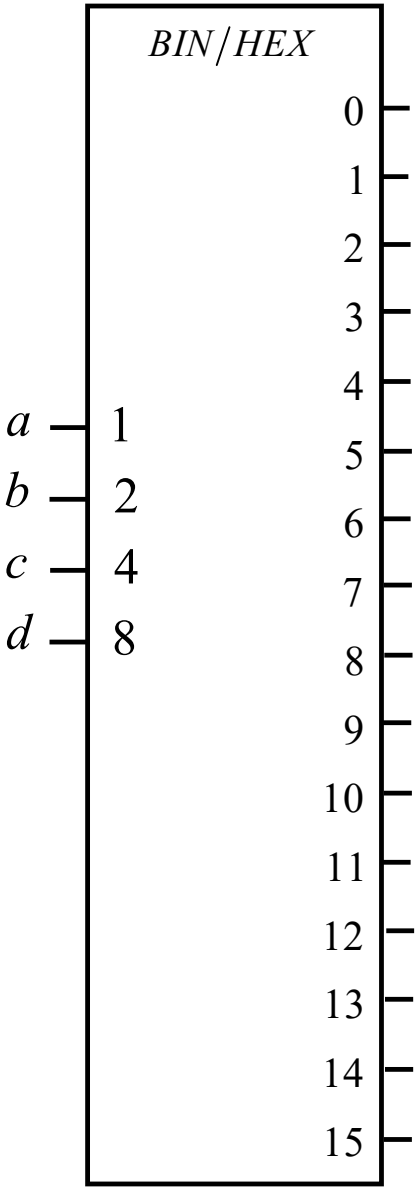
8	4	2	1										
d	c	b	a	$\overline{P}_9$	$\overline{P}_8$	$\overline{P}_7$	$\overline{P}_6$	$\overline{P}_5$	$\overline{P}_4$	$\overline{P}_3$	$\overline{P}_2$	$\overline{P}_1$	$\overline{P}_0$
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0	1	1	1	1	1
0	1	1	0	1	1	1	0	1	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	1	1	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1	1	1	1	1
1	0	1	0	1	1	1	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

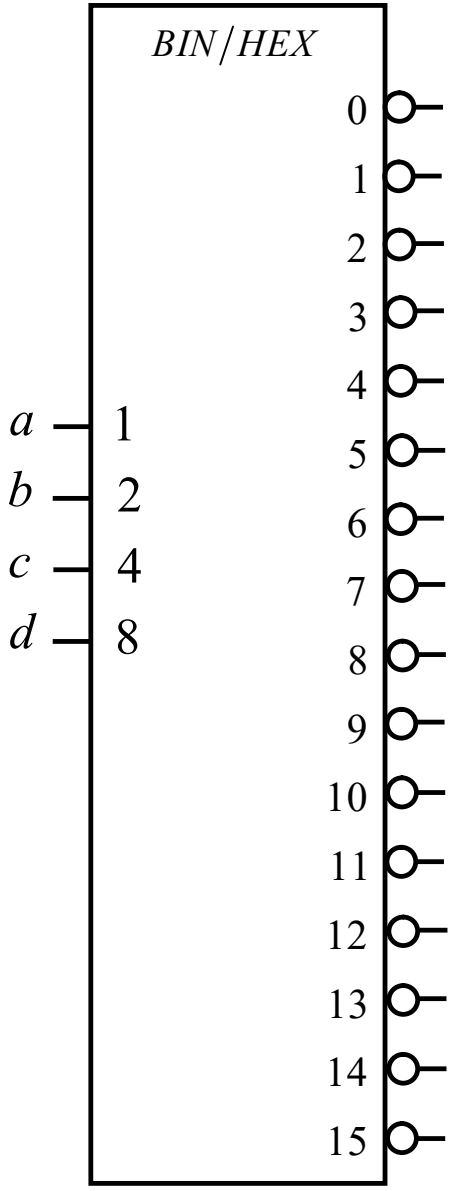
Se pone a 1 el estado lógico interno de la salida que tiene asignado un número igual a la suma de los pesos de las entradas que están a 1. Los pesos asociados a las entradas *d*, *c*, *b* y *a* son 8, 4, 2 y 1 respectivamente. El mayor valor que se puede decodificar es el 1001_2

Ejemplos:

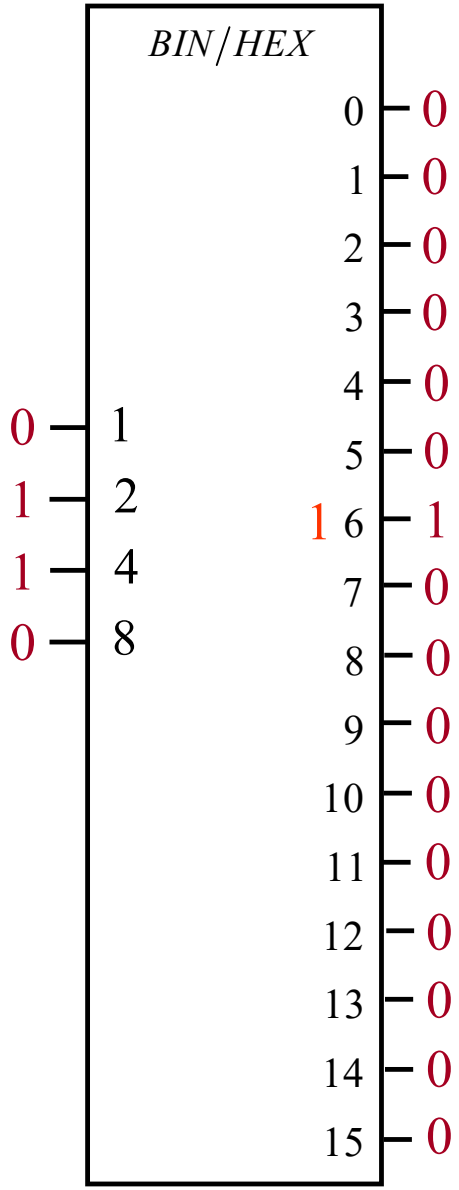

BCD/DEC
0 1 0 2 0 4 0 8
1 0 0 1 0 2 0 3 0 4 0 5 0 6 0 7 0 8 0 9
0 1 1 1 1 1 1 1 1 1
BCD/DEC
0 1 0 2 1 4 0 8
0 0 0 1 0 2 0 3 1 4 0 5 0 6 0 7 0 8 0 9
1 1 1 1 1 1 0 1 1 1
BCD/DEC
1 1 1 2 1 4 0 8
0 0 0 1 0 2 0 3 0 4 0 5 0 6 1 7 0 8 0 9
1 1 1 1 1 1 1 0 1 1
BCD/DEC
1 1 1 2 0 4 1 8
0 0 0 1 0 2 0 3 0 4 0 5 0 6 0 7 0 8 0 9
1 1 1 1 1 1 1 1 1 1

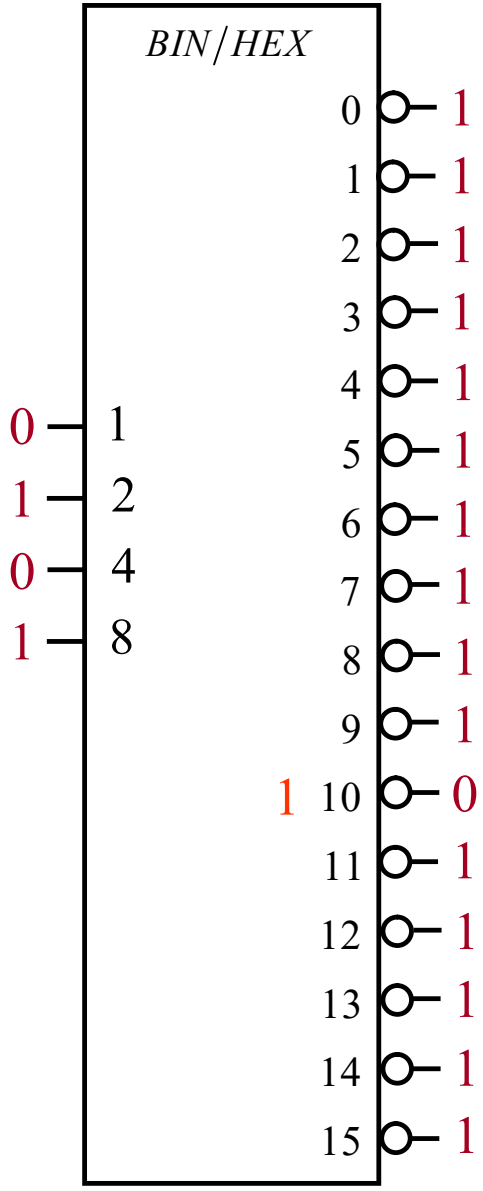
Decodificador hexadecimal





Ejemplos:


BIN/HEX
0 1
1 2
1 4
0 8
0 1 2 3 4 5 1 6 7 8 9 10 11 12 13 14 15
0 0 0 0 0 1 0 0 0 0 0 0 0 0 0

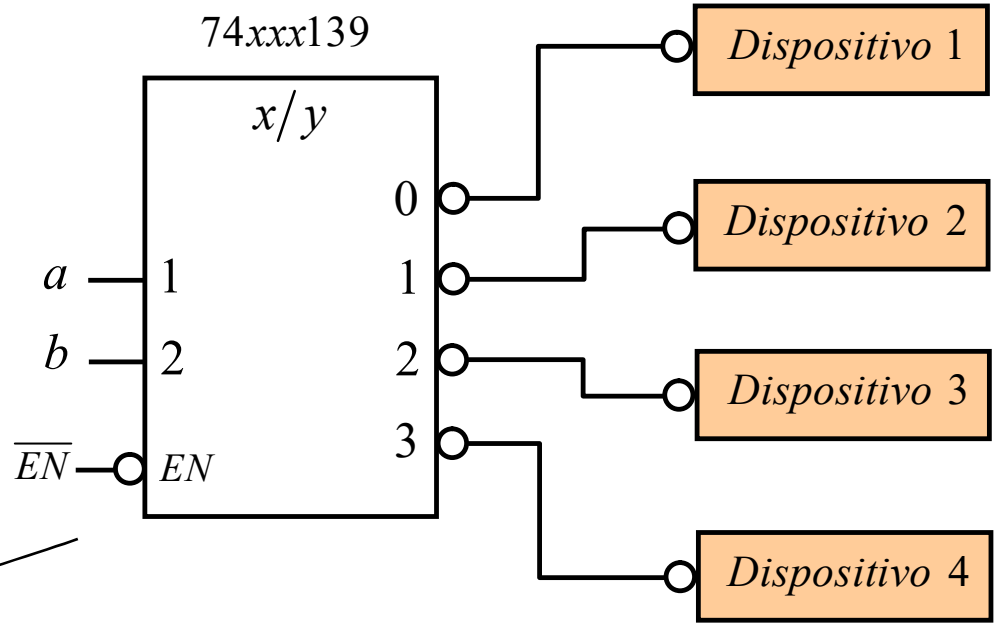

BIN/HEX
0 1
1 2
0 4
1 8
0 1 2 3 4 5 6 7 8 9 1 10 11 12 13 14 15
1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1

Aplicaciones:

1ª: Decodificar palabras código (ver diapositivas anteriores).

2ª: Control del funcionamiento no simultáneo de varios dispositivos.

Ejemplo:



con dos señales binarias (*a* y *b*) se puede controlar el funcionamiento de hasta 4 dispositivos

3ª: Construir decodificadores de otros tipos de códigos: por ejemplo, utilizando decodificadores decimales se puede construir:

_ Un decodificador de cualquier código *BCD* (*Aiken*, *exceso*-3, etc)

_ Un decodificador hexadecimal

_ etc.

4ª: Implementar funciones lógicas (en algunos casos se consigue reducir el número de ICs a utilizar).

Ejemplo 1: Diseñar un circuito que implemente las siguientes funciones, utilizando un decodificador adecuado y puertas NAND

$$f_1(c,b,a) = \sum_3 (0,1,3,5)$$

$$f_2(c,b,a) = \sum_3 (5,6,7) + \sum_\phi (1,4)$$

Ejemplo 2: Diseñar un circuito que implemente la siguiente función, utilizando un decodificador decimal y puertas NAND

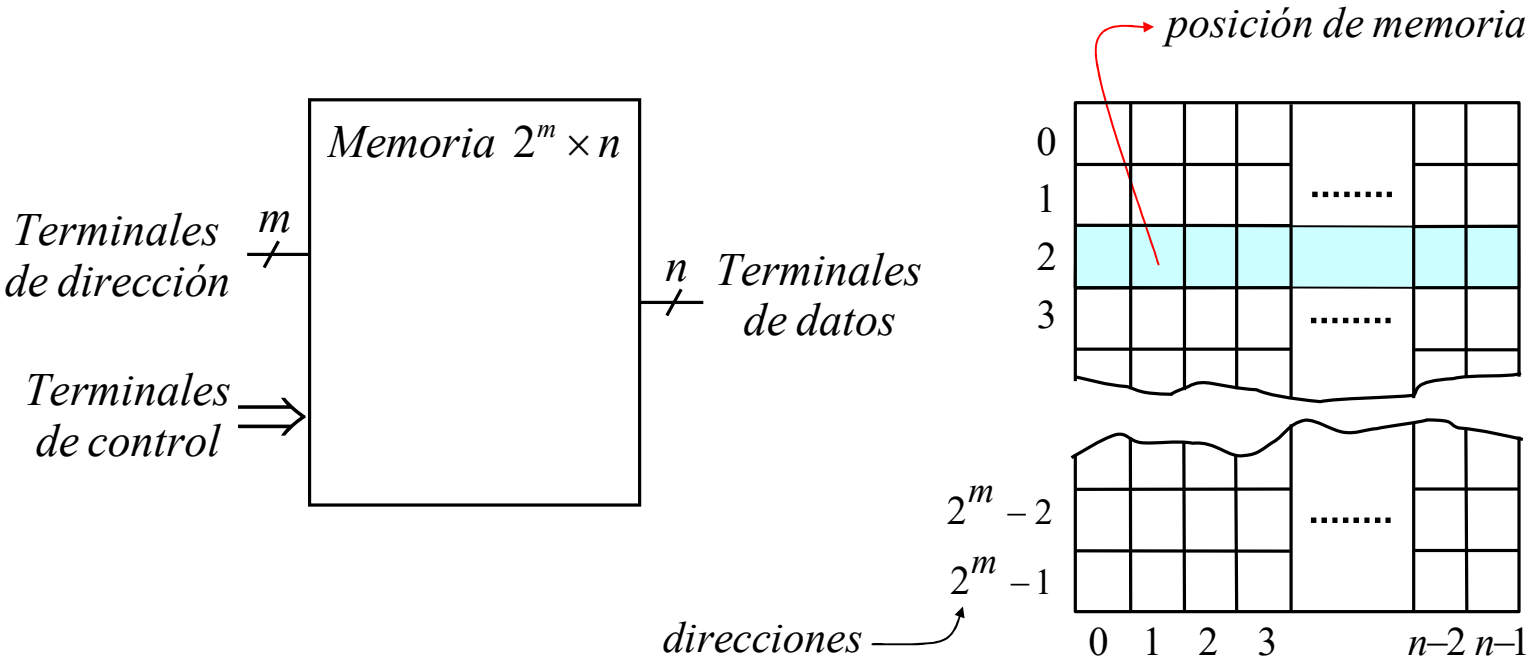
$$g(b,a) = \sum_2 (0,1,2)$$

Ejemplo 3: Diseñar un circuito que implemente la siguiente función, utilizando un decodificador adecuado y el menor número posible de puertas NAND de 2 entradas.

$$h(d,c,b,a) = \overline{\overline{\overline{d} \cdot c \cdot \overline{a}} \cdot (c + b + \overline{a})}$$

no

5ª: Decodificar las direcciones presentes en el bus de direcciones de una memoria

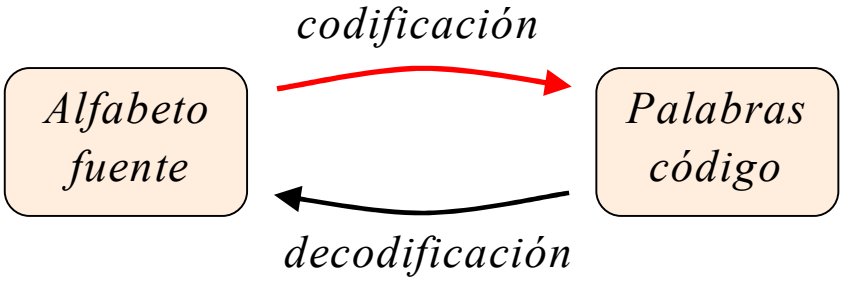


_ La memoria tiene 2^m posiciones. Cada posición tiene asociada una dirección. La posición en la que se lee o se escribe se determina decodificando la dirección presente en los terminales de dirección (*bus de direcciones*).

_ En cada posición (≡ dirección de memoria) se guardan n bits (≡ 1 *palabra* ≡ 1 *word*)

▪ Codificadores de *n* líneas de entrada a *m* líneas de salida

DEF.: Un codificador es un circuito combinacional que proporciona para cada símbolo de un alfabeto fuente dado la palabra código que le corresponde según el código que se considere.



Hay dos tipos de codificadores: *con prioridad* y *sin prioridad*. Su funcionamiento es el mismo excepto en el caso de que se activen simultáneamente dos o más entradas. Los codificadores *sin prioridad* no se utilizan.

Palabra código
DECODIFICADOR
Símbolo

Símbolo
CODIFICADOR
Palabra código

Ejemplo:

Alfabeto Fuente	Palabras código $BCD_{natural}$	Alfabeto código
0	0 0 0 0	0
1	0 0 0 1	1
2	0 0 1 0	
3	0 0 1 1	
4	0 1 0 0	
5	0 1 0 1	
6	0 1 1 0	
7	0 1 1 1	
8	1 0 0 0	
9	1 0 0 1	

Los codificadores que se estudian en esta asignatura proporcionan la palabra código correspondiente a la entrada activa que tiene el mayor valor asociado (*codificadores con prioridad*).

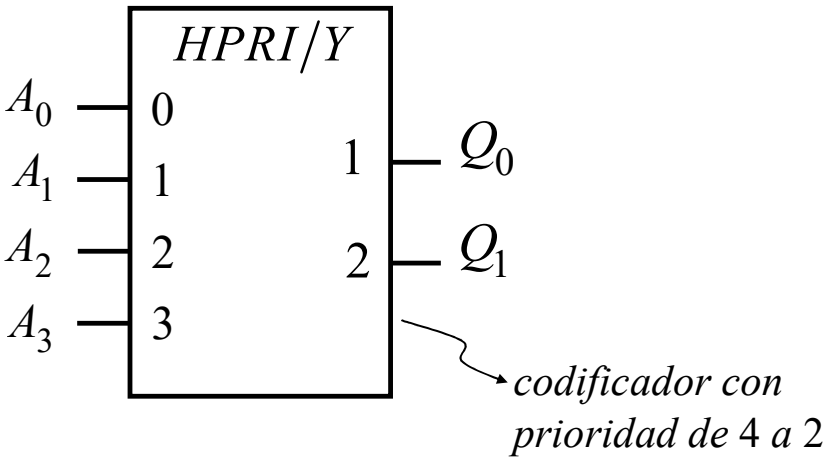
Nota: un codificador realiza la función inversa de la que realiza un decodificador. El valor presente en sus salidas indica la entrada que activa que tiene asignado un valor mayor.

Codificadores con prioridad (o de *alta prioridad* ≡ *high priority*): cuando se activan simultáneamente dos o más entradas, en la salida aparece la codificación correspondiente a la entrada activa que tenga asignado el valor más alto.

Curiosidad: una de las aplicaciones de los codificadores con prioridad es la de gestionar el acceso a recursos compartidos. Si se reciben dos o más peticiones de uso de un mismo recurso, es un codificador con prioridad el que decide cuál de las solicitudes tiene mayor prioridad y, por lo tanto, la que accede primero al recurso solicitado.

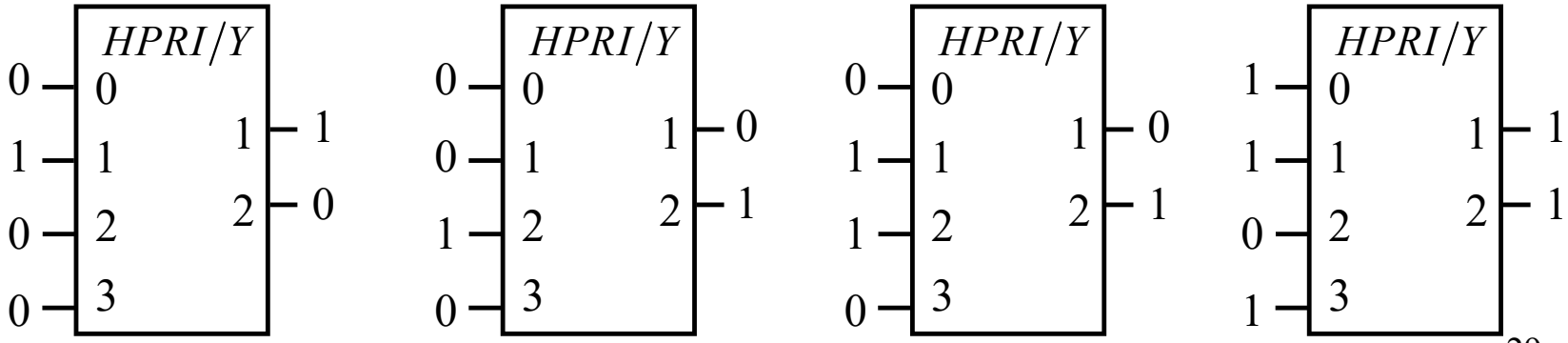
En microcontroladores avanzados, en DSPs y en DSCs cuando se activa el *flag* de dos o más fuentes de interrupción, se utiliza un codificador con prioridad para determinar la fuente de interrupción que la CPU atiende en primer lugar.

Codificadores con prioridad (o de *alta prioridad* ≡ *high priority*): cuando se activan simultáneamente dos o más entradas, en la salida aparece la codificación (*palabra código*) correspondiente a la entrada activa que tenga asignado el valor más alto.

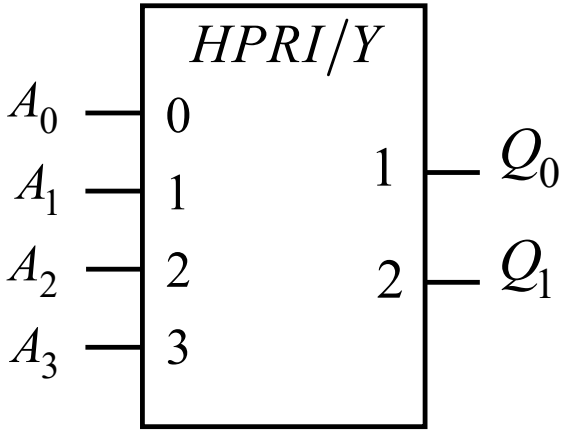


A_3	A_2	A_1	A_0	Q_1	Q_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1
	⋮			⋮	
0	1	1	0	1	0
	⋮			⋮	
0	0	0	0	0	0

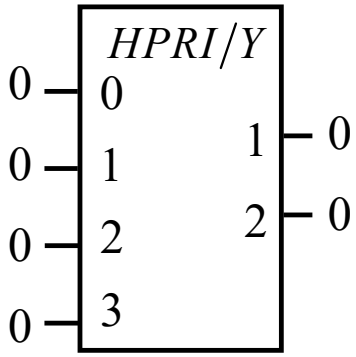
Ejemplos:

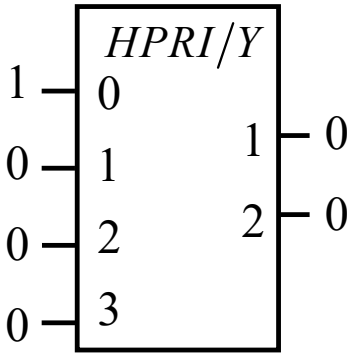


Problema: observando las salidas no se puede distinguir entre la situación en la que sólo está activa la entrada que tiene asignado el valor 0 y la situación en la que no hay ninguna entrada activa.



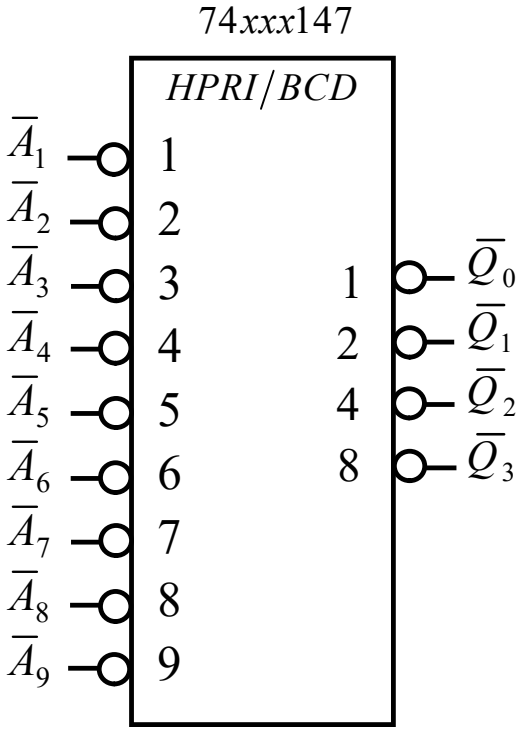
A_3	A_2	A_1	A_0	Q_1	Q_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1
	⋮			⋮	
0	1	1	0	1	0
	⋮			⋮	
0	0	0	0	0	0





1ª *solución*: Omitir la entrada de menor peso/prioridad

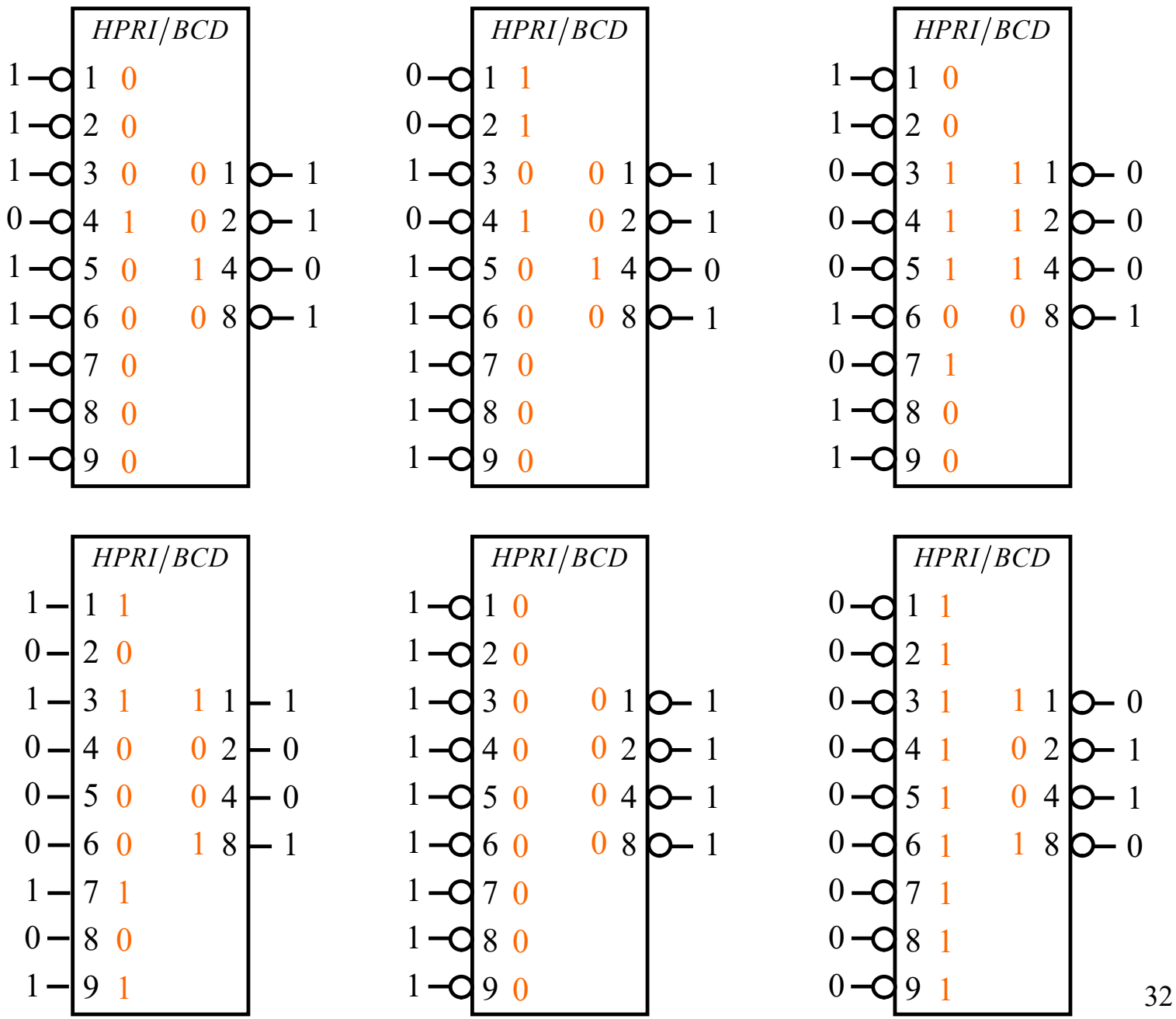
Ejemplo: Codificador con prioridad de Decimal a $BCD_{natural}$ ó de 9 a 4



$\overline{A}_9$	$\overline{A}_8$	$\overline{A}_7$	$\overline{A}_6$	$\overline{A}_5$	$\overline{A}_4$	$\overline{A}_3$	$\overline{A}_2$	$\overline{A}_1$	$\overline{Q}_3$	$\overline{Q}_2$	$\overline{Q}_1$	$\overline{Q}_0$
0	x	x	x	x	x	x	x	x	0	1	1	0
1	0	x	x	x	x	x	x	x	0	1	1	1
1	1	0	x	x	x	x	x	x	1	0	0	0
1	1	1	0	x	x	x	x	x	1	0	0	1
1	1	1	1	0	x	x	x	x	1	0	1	0
1	1	1	1	1	0	x	x	x	1	0	1	1
1	1	1	1	1	1	0	x	x	1	1	0	0
1	1	1	1	1	1	1	0	x	1	1	0	1
1	1	1	1	1	1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1

Las *entradas* están numeradas consecutivamente, en este caso, empezando en 1. Las *salidas* tienen asignados *pesos*. Los *estados lógicos internos* de las salidas toman, de acuerdo con los pesos que tienen asignados, el valor correspondiente al número más alto de todas las entradas que estén activas (entradas cuyo estado lógico interno sea igual a 1)

Ejemplos:


HPRI/BCD
1 1 1 0 1 1 1 0 1 1 1 1
1 2 1 0 1 1 1 0 1 1 1 1
1 3 1 0 0 1 1 0 1 1 0 0
0 4 0 1 0 2 1 1 1 0 0 0
1 5 1 0 1 4 0 1 1 0 0 1
1 6 1 0 0 8 1 0 1 1 1 1
1 7 1 0 1 0 1 1 0 1 0 0
1 8 1 0 1 0 1 1 0 1 1 1
1 9 1 0 1 0 1 1 0 1 1 1

▪ Multiplexores de n canales (de entrada) o selector de datos.

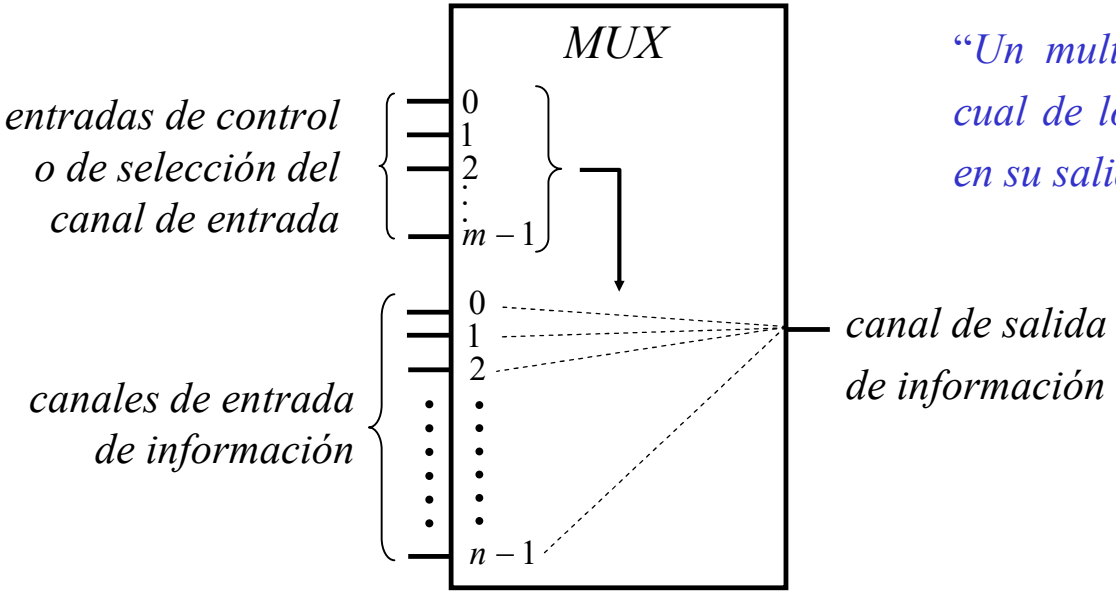
DEF.: Un multiplexor es un circuito combinacional que tiene:

_ 1 salida (canal) de información.

_ m entradas de control o de selección (de uno de los canales de entrada).

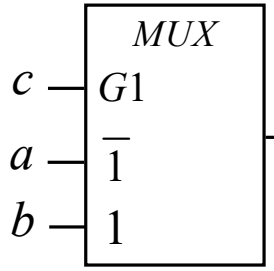
_ n entradas (canales) de información, cumpliéndose que: $2^m = n$

Su funcionamiento se caracteriza por lo siguiente: "*La combinación binaria aplicada a las* m *entradas de control selecciona el canal de entrada cuya información lógica* (1 *ó* 0) *aparece en el canal de salida*"



"*Un multiplexor permite seleccionar cual de los datos de entrada aparece en su salida*"

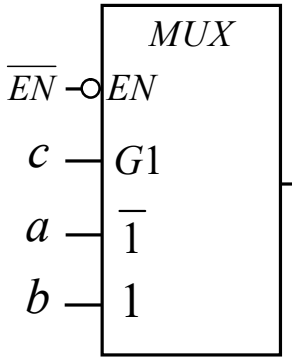
Multiplexor de 2 canales



$$f(c,b\ a) = \overline{c}\ a + c\ b$$

c	f
0	a
1	b

→ f vale 1 si c vale 0 y a vale 1 o bien si c vale 1 y b vale 1

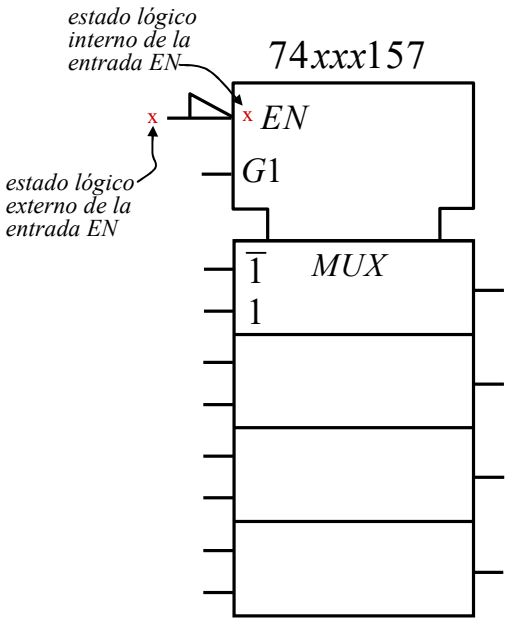


$$f(\overline{EN},c,b,a) = \overline{\overline{EN}}\cdot\overline{c}\cdot a + \overline{\overline{EN}}\cdot c\cdot b = EN(\overline{c}\cdot a + c\cdot b)$$

$\overline{EN}$	c	b	a	f
1	x	x	x	0
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1

Nota: *EN* ≡ *enable* ≡ *entrada de habilitación*. Cuando su estado lógico interno es 1 el circuito funciona normalmente mientras que cuando es 0 su funcionamiento está inhibido o deshabilitado.

Cuádruple multiplexor de 2 canales: los cuatro multiplexores comparten la entrada de selección y la entrada de inhibición (*En*: *enable*)

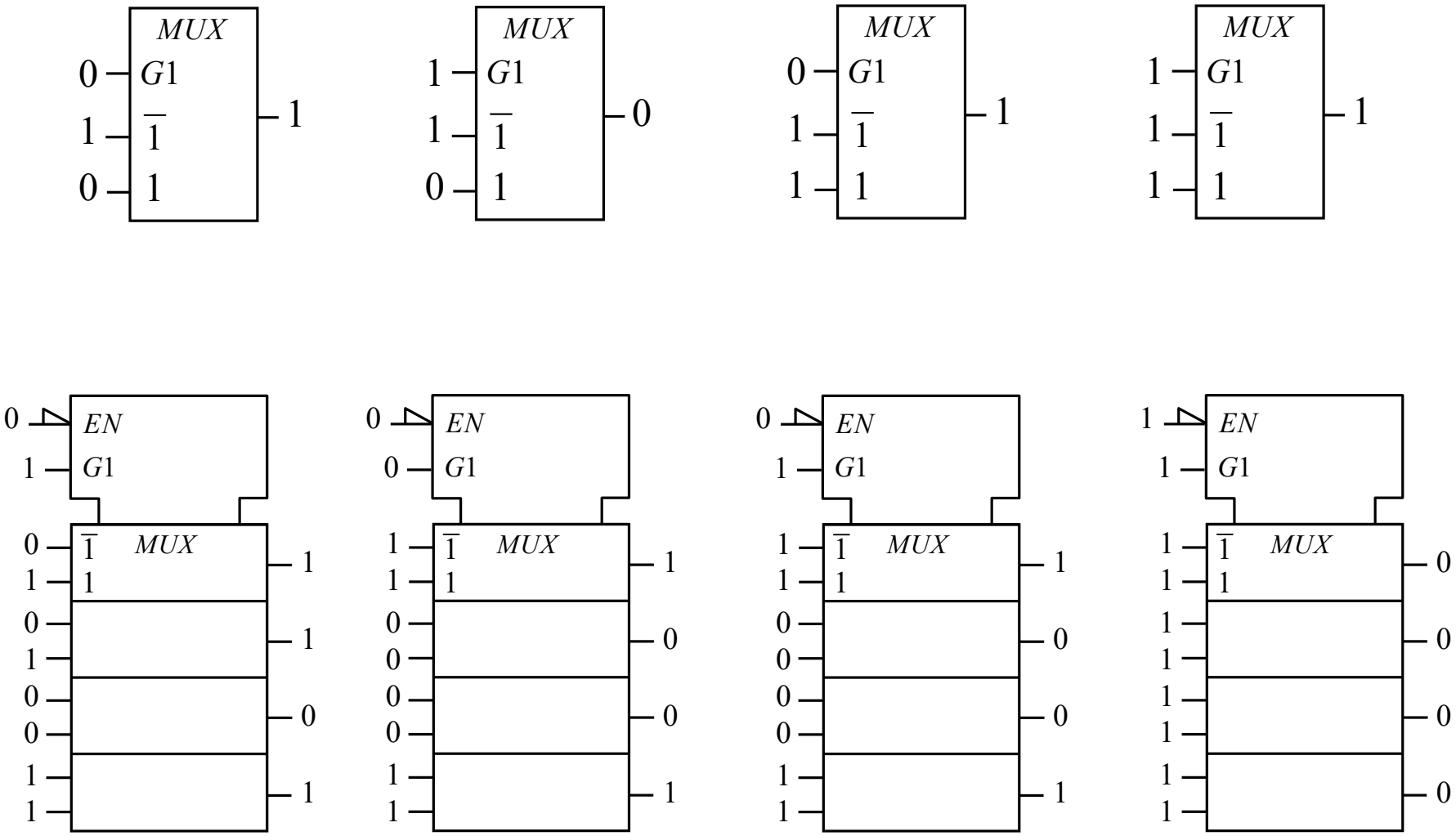


Nota: cuando en una entrada de habilitación, después de *EN* no hay nada (un número o una letra), y las salidas no se pueden poner en *tercer estado* (no tienen el símbolo ▽), se cumple lo siguiente:

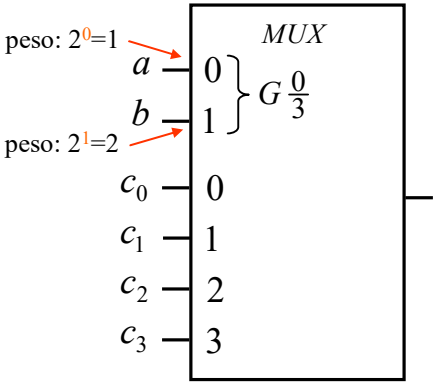
· Si el *estado lógico interno* de la entrada *EN* es igual a 1, entonces el circuito funciona normalmente.

· Si el *estado lógico interno* de la entrada *EN* es igual a 0, entonces los *estados lógicos internos* de todas las salidas del circuito se ponen a 0, con independencia de los valores que tomen las demás entradas.

Ejemplos:


MUX
G1
1
0
1
0
1
MUX
G1
1
1
0
1
0
MUX
G1
1
0
1
1
1
MUX
G1
1
1
1
1
1
EN
G1
0
1
MUX
1
1
0
1
0
0
1
1
EN
G1
0
0
MUX
1
1
0
0
0
0
1
1
EN
G1
0
1
MUX
1
1
0
0
0
0
1
1
EN
G1
1
1
MUX
1
1
0
0
0
0
1
1

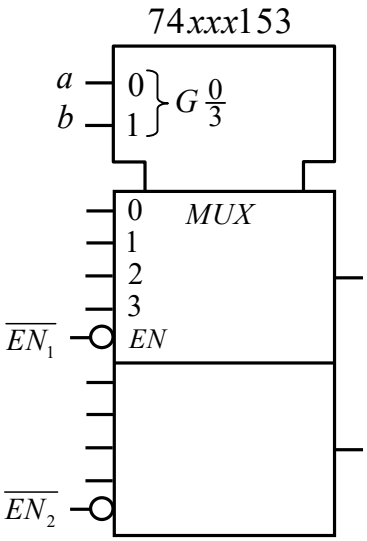
Multiplexor de 4 canales



$$f(c_3, c_2, c_1, c_0, b, a) = c_0 \bar{b}\,\bar{a} + c_1 \bar{b}\,a + c_2 b\,\bar{a} + c_3 b\,a$$

b	***a***	f
0	0	c_0
0	1	c_1
1	0	c_2
1	1	c_3

Doble multiplexor de 4 canales:

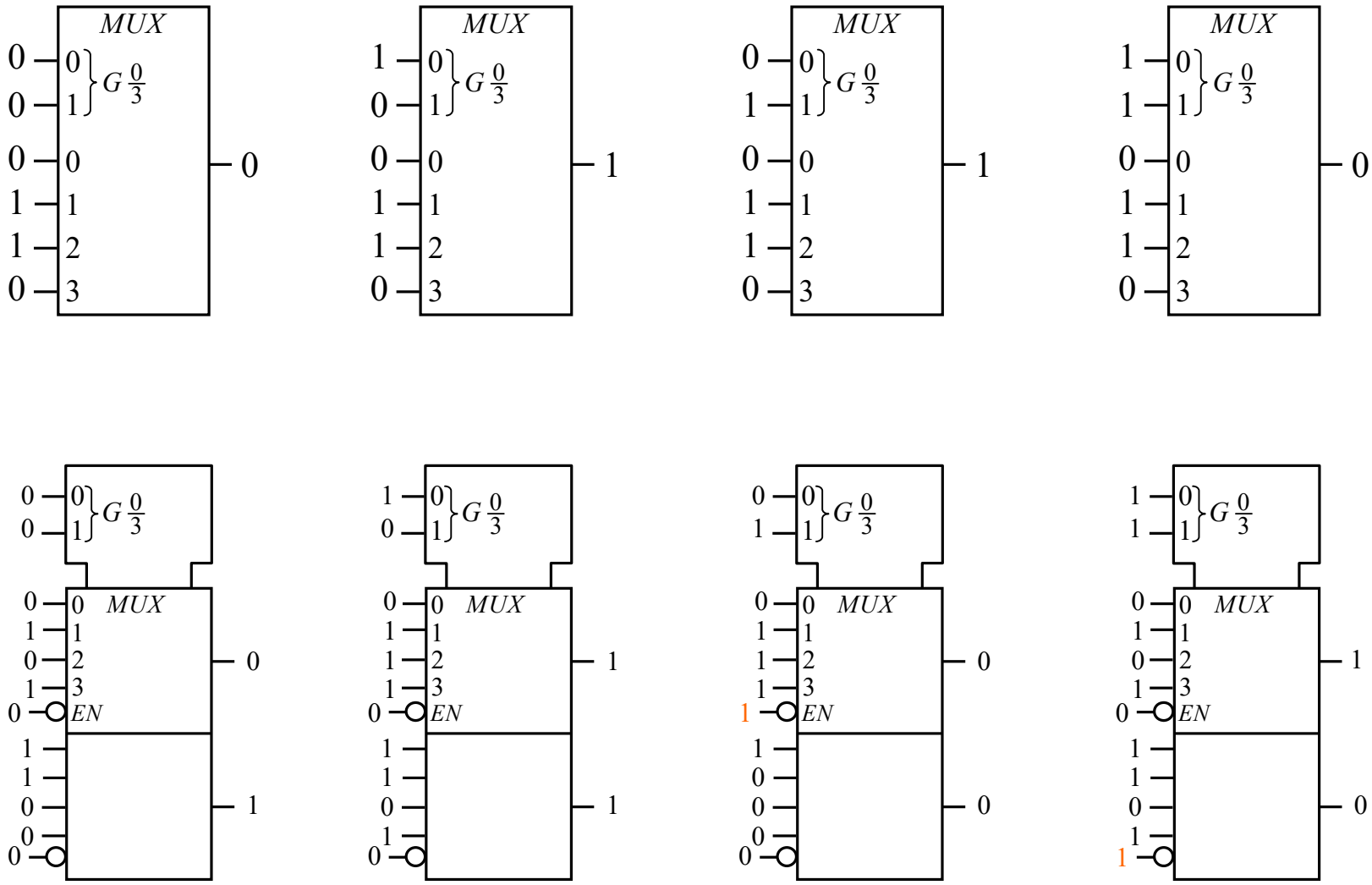


Nota: en este circuito se cumple lo siguiente:

_ Los multiplexores se habilitan/deshabilitan de forma independiente

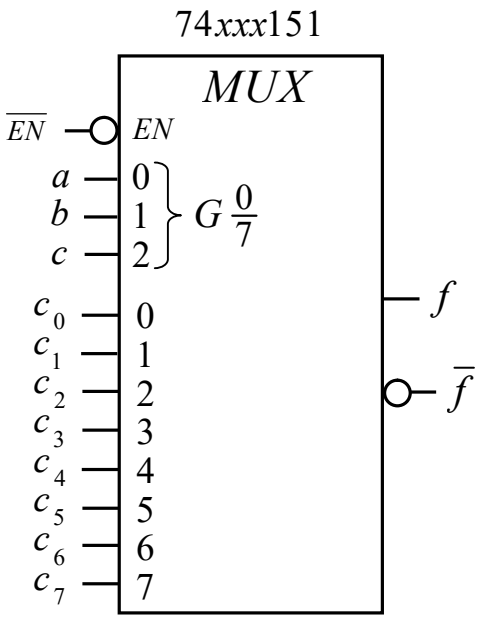
_ Ambos multiplexores comparten las entradas de selección (b y a)

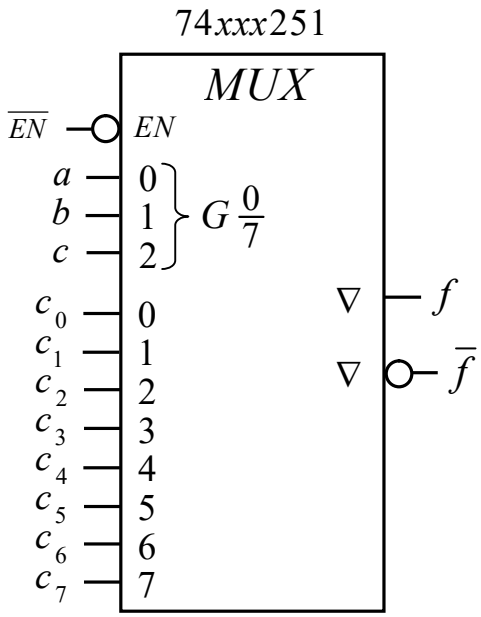
Ejemplos:



Multiplexores de 8 canales

Nota: cuando una salida lleva el símbolo ▽ significa que dicha salida puede ponerse en un *tercer estado* (≡ estado de muy alta impedancia), además de poder ponerse en los dos estados normales: 1 y 0.





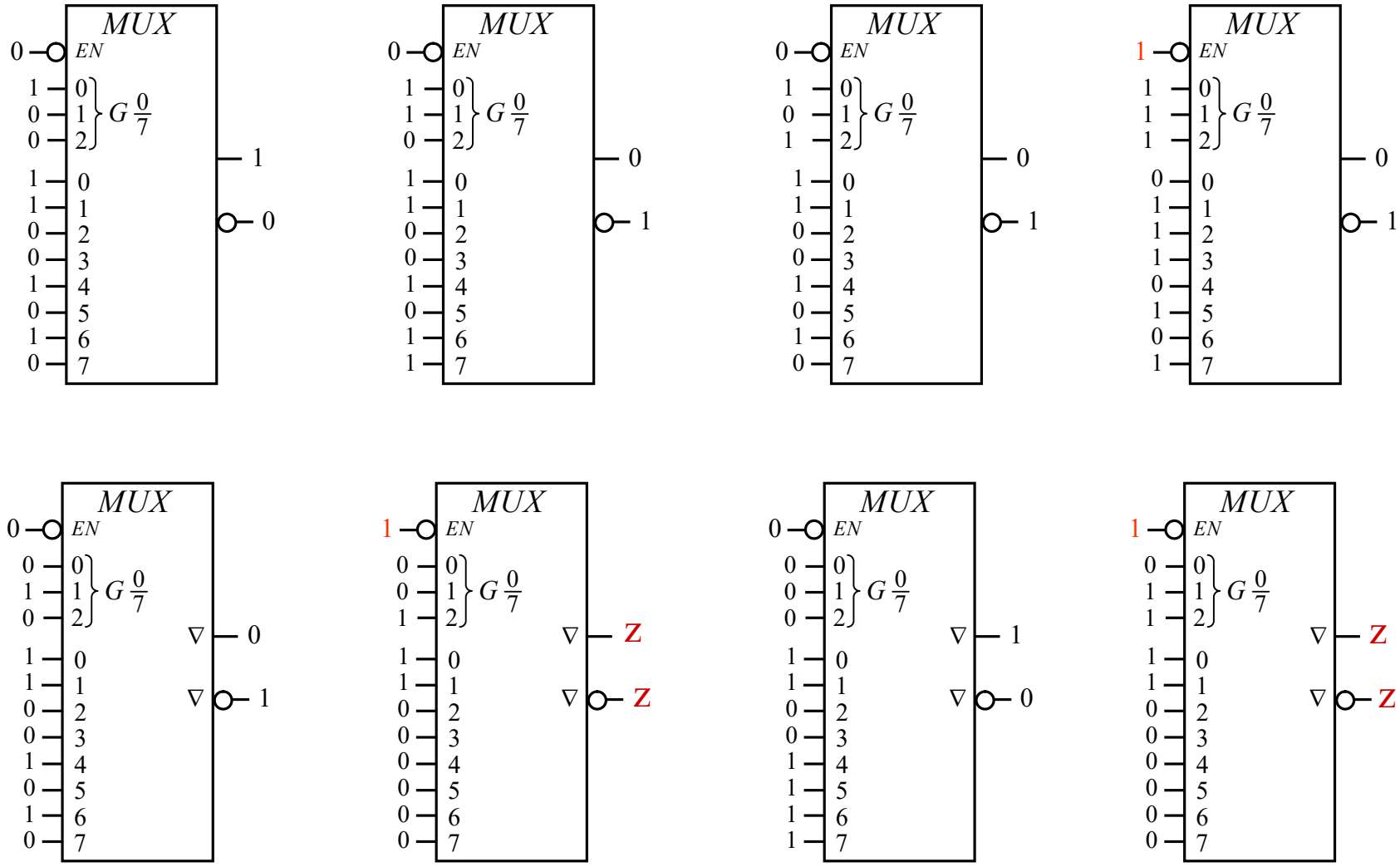
Nota: cuando después del símbolo calificador *EN* no hay ni un número ni una letra (no hay nada) y las salidas pueden ponerse en un *tercer estado* (▽), se cumple lo siguiente:

· Si el *estado lógico interno* de la entrada *EN* es igual a 1, entonces el circuito funciona normalmente.

· Si el *estado lógico interno* de la entrada *EN* es igual a 0, entonces las salidas se ponen en el *tercer estado*, con independencia de los valores que tomen las demás entradas del circuito

· *Tri-state is a trademark of National Semiconductor Corporation. Their very expensive lawyer thought you'd like to know it* (*tri-sate ≡ three-sate*).

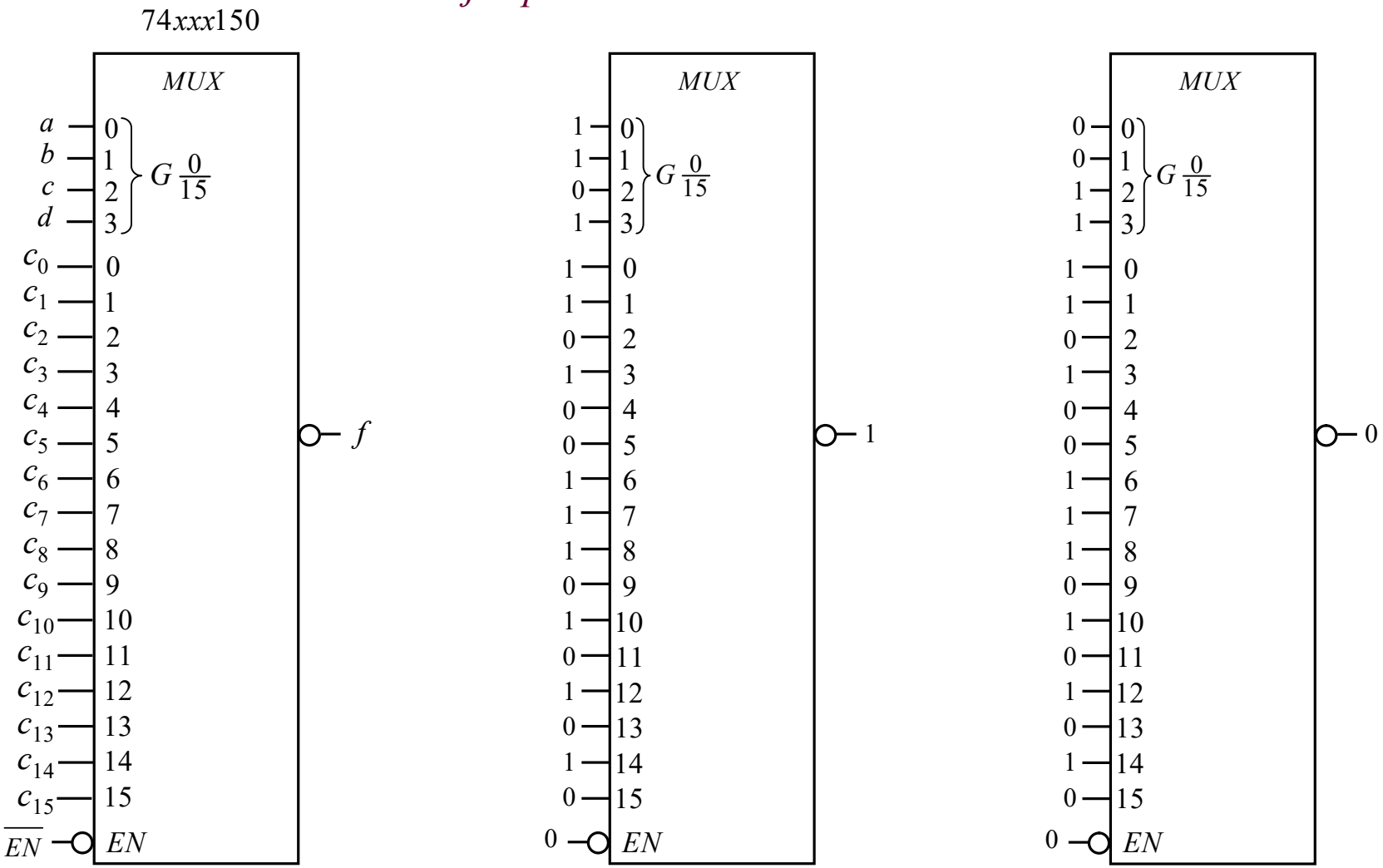
Ejemplos:



Nota: aquí se utiliza la letra *z* para indicar que una salida está en tercer estado.

Multiplexor de 16 canales

Ejemplos:



Aplicación: Implementación de funciones lógicas con multiplexores

- Cualquier función dependiente de n variables, *siempre* se puede implementar utilizando:

opción 1: Un multiplexor de n entradas de selección (≡ un multiplexor de 2^n canales)

opción 2: Un multiplexor de n–1 entradas de selección y 1 puerta NOT.

- Algunas funciones lógicas dependientes de n variables se pueden implementar utilizando únicamente un multiplexor de n–1 variables de selección (≡ un multiplexor de 2^{n-1} canales)

Nota 1: el objetivo es implementar funciones utilizando el menor número posible de circuitos integrados (ICs) y de componentes.

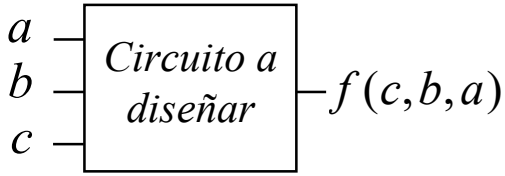
Nota 2: *Xilinx* utiliza memorias para implementar funciones en sus *FPGAs* (ver tema 6), mientras que *Actel* (ahora *Microsemi*) utiliza multiplexores (ver las siguientes diapositivas)

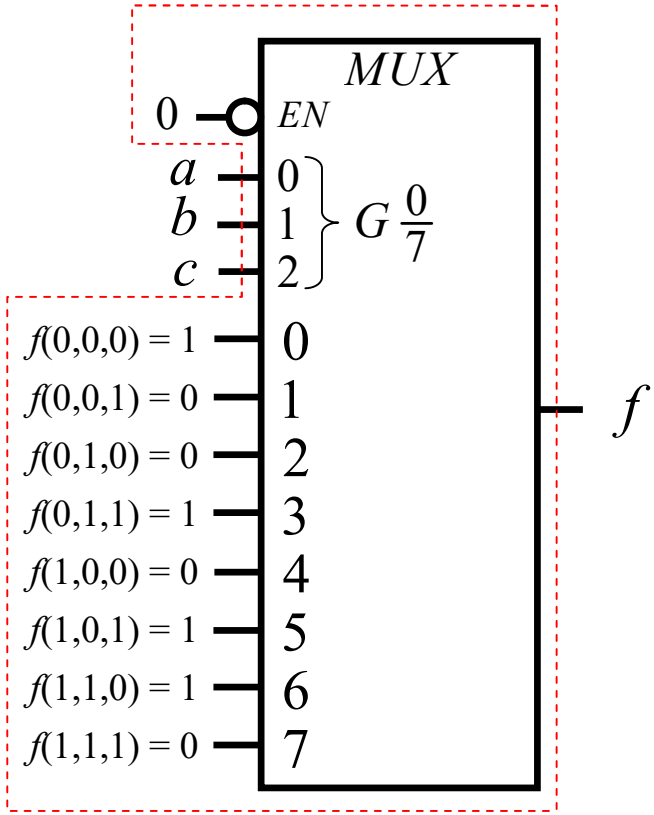
Nota 3: las *LUT* (*look-up table*) de las *logic cells* son las que implementan las funciones lógicas (circuitos combinacionales).

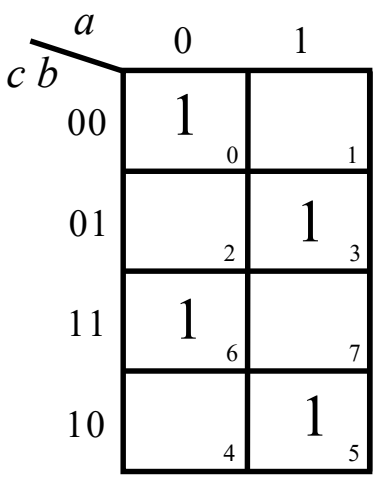
Nota 4: *FPGA*: *field programmable gate array*

Ejemplo 1: no se dispone de ninguna de las variables de las que depende la función tanto en su forma directa como negada.

$$f(c,b,a) = \sum_3 (0,3,5,6)$$





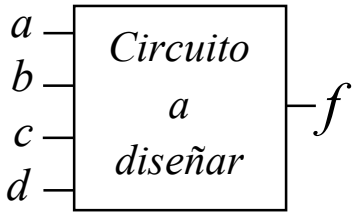


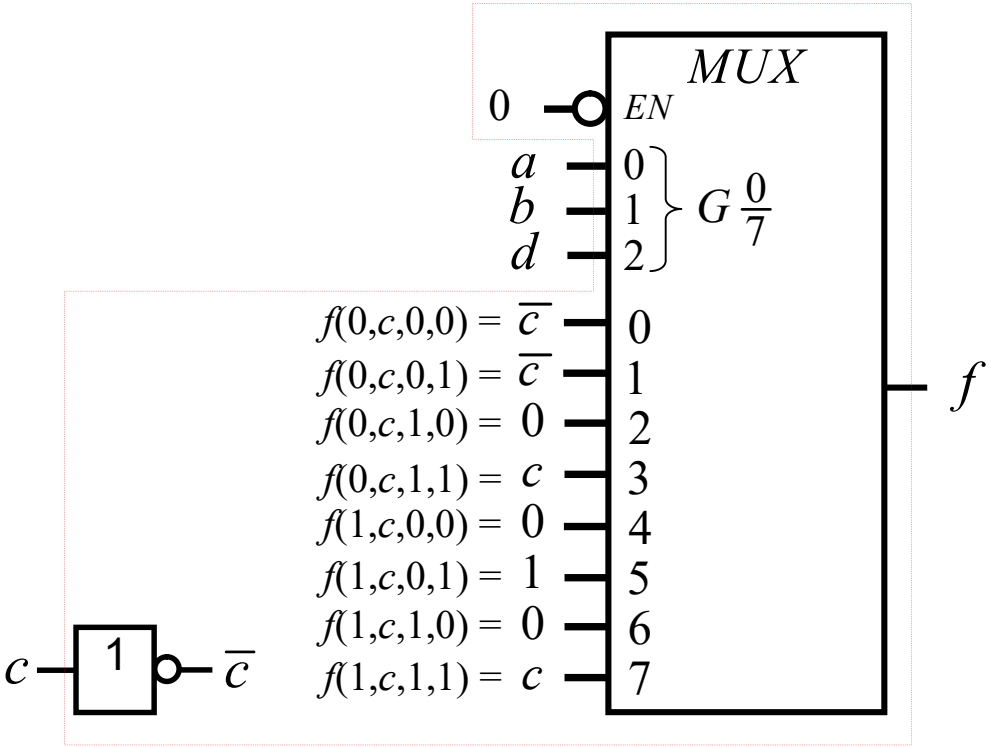
c	b	a	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

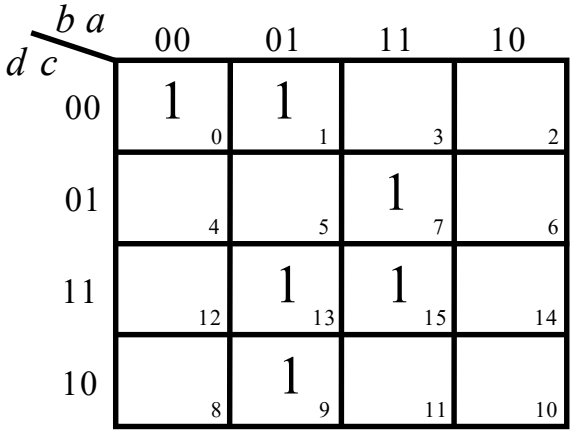
Ejemplo 3: en este caso, los valores que hay que poner en los canales de entrada del multiplexor se determinan utilizando una tabla de *Karnaugh-Veitch*.

Solución 1:

$$f(d,c,b,a) = \sum_4 (0,1,7,9,13,15)$$

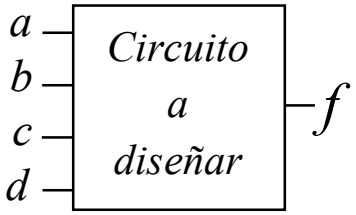


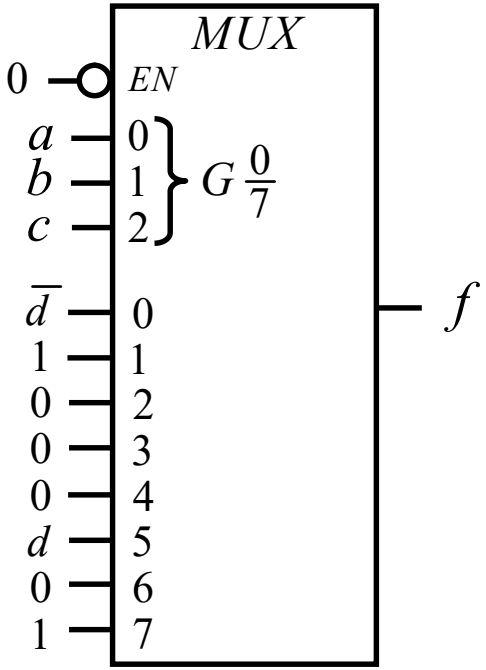


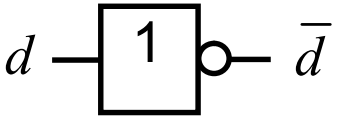


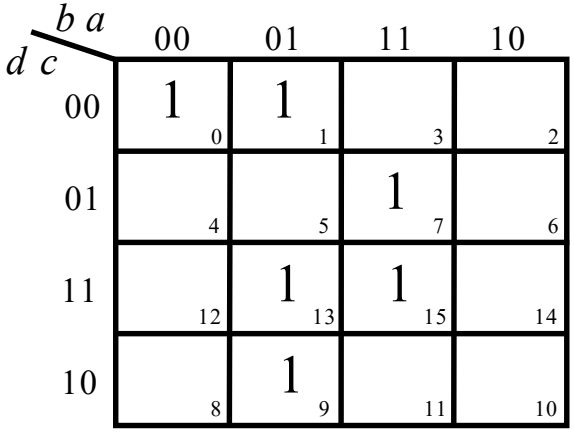
Solución 2: en las entradas de selección del multiplexor se pone un conjunto de variables distinto al de la solución anterior.

$$f(d,c,b,a)=\sum_4(0,1,7,9,13,15)$$

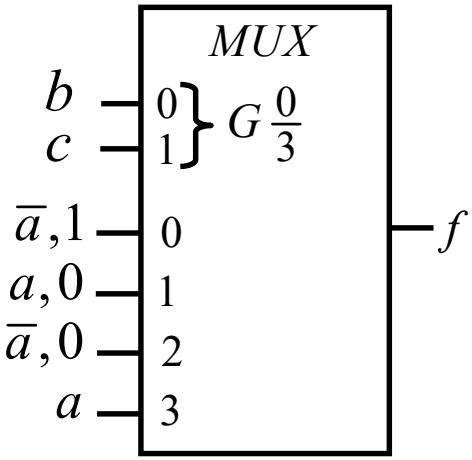


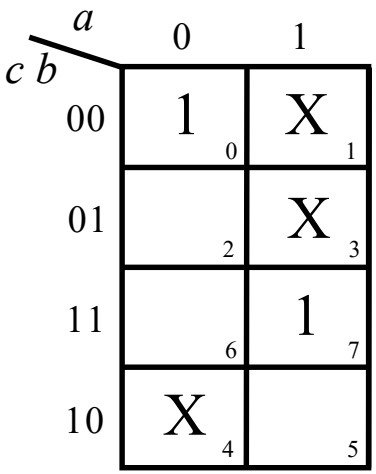


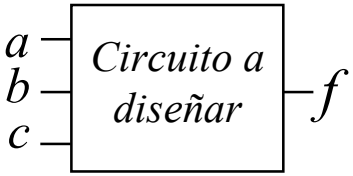




Ejemplo 3: la función no está totalmente definida $f(c,b,a)=\sum_3(0,7)+\sum_\phi(1,3,4)$







c	b	a	f	
0	0	0	1	$\overline{a},1$
0	0	1	x	
0	1	0	0	$0, a$
0	1	1	x	
1	0	0	x	$0,\overline{a}$
1	0	1	0	
1	1	0	0	a
1	1	1	1	

Pregunta: En la práctica, ¿qué *valores-señales* se ponen en los canales de entrada?

- Demultiplexores de n canales (de salida)

DEF.: Un demultiplexor es un circuito combinacional que tiene:

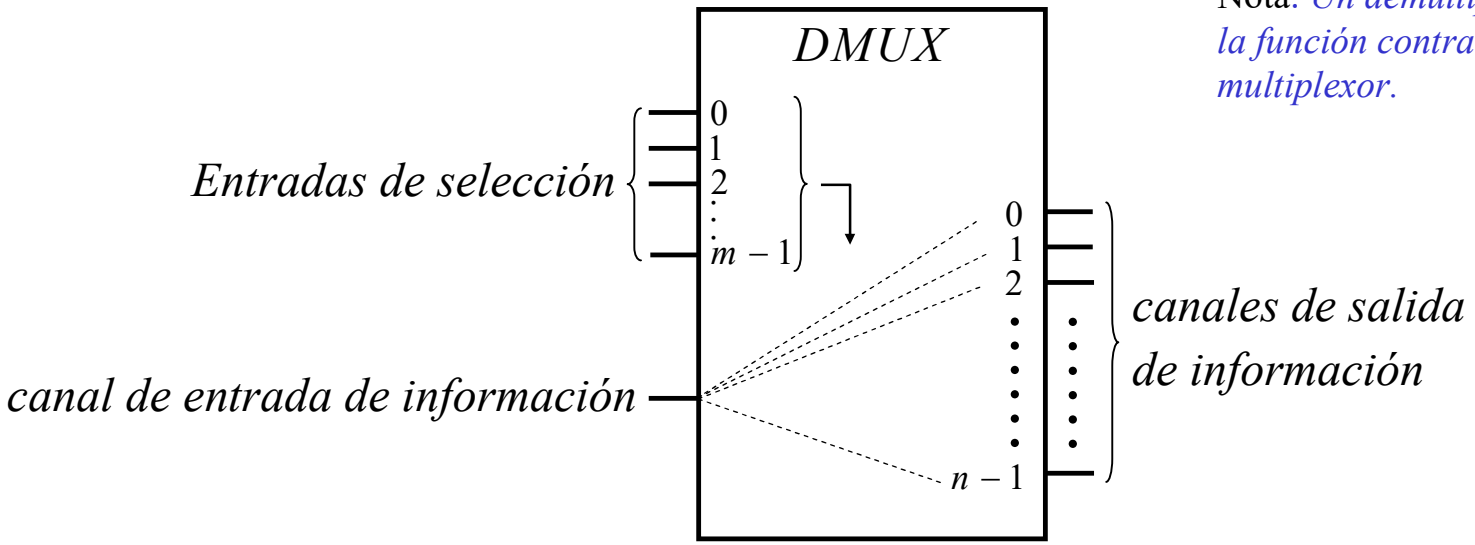
_ 1 entrada (canal) de *información*.

_ m entradas de *control* o de *selección*.

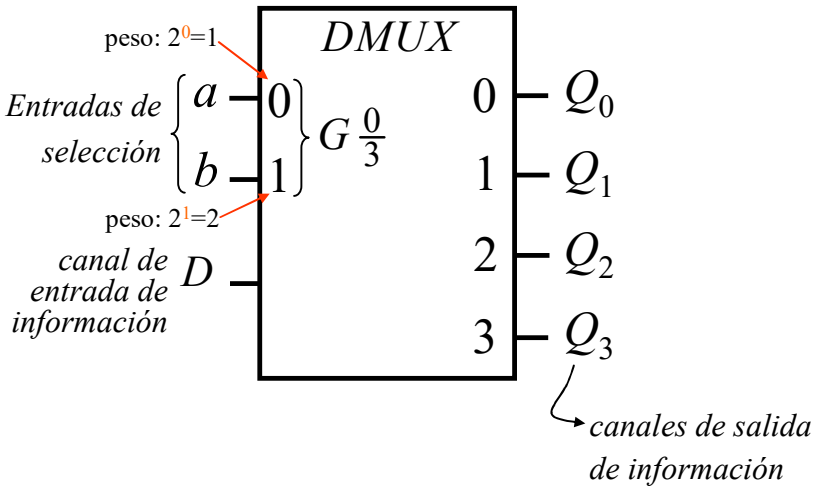
_ n salidas (canales) de información, cumpliéndose que: $2^m = n$

Su funcionamiento se caracteriza por lo siguiente: " *La combinación binaria aplicada a las m entradas de control selecciona el canal de salida en el que aparece el valor* (1 *ó* 0) *presente en el canal de entrada"*

Nota: *Un demultiplexor realiza la función contraria a la de un multiplexor.*



Demultiplexor de 4 canales (de salida)



entrada de información — *entradas de selección*

D	b	a	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0	0
1	0	0	0	0	0	1
0	0	1	0	0	0	0
1	0	1	0	0	1	0
0	1	0	0	0	0	0
1	1	0	0	1	0	0
0	1	1	0	0	0	0
1	1	1	1	0	0	0

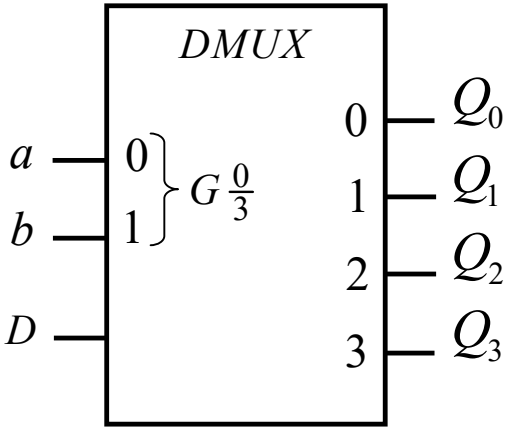
|||

b	a	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

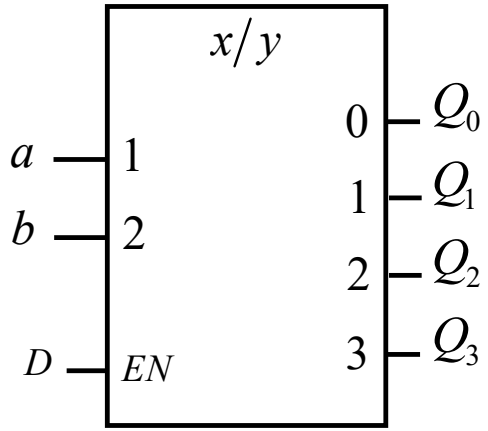
Cuando (b a) = 0 0, en la salida Q_0 hay el valor lógico que haya en la entrada de información D y en las demás salidas hay un 0.

no

Se fabrican circuitos que pueden funcionar como *demultiplexores* y como *decodificadores*



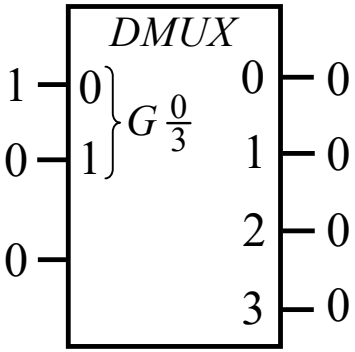
D	b	a	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0	0
1	0	0	0	0	0	1
0	0	1	0	0	0	0
1	0	1	0	0	1	0
0	1	0	0	0	0	0
1	1	0	0	1	0	0
0	1	1	0	0	0	0
1	1	1	1	0	0	0

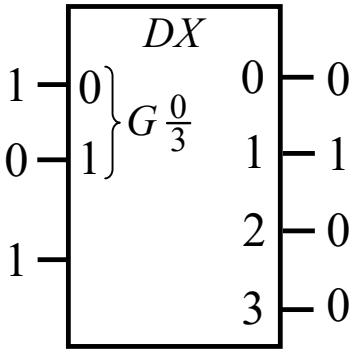


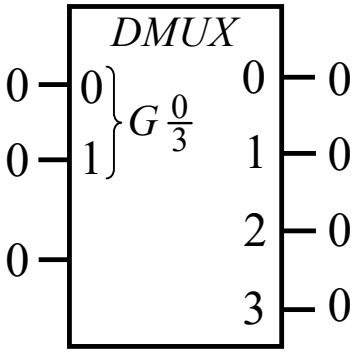
(*decodificador*)

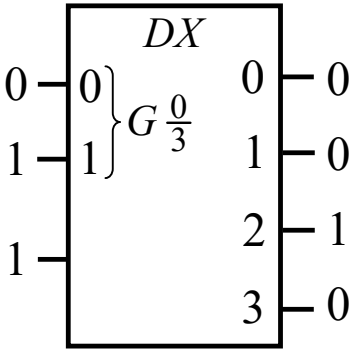
D	b	a	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

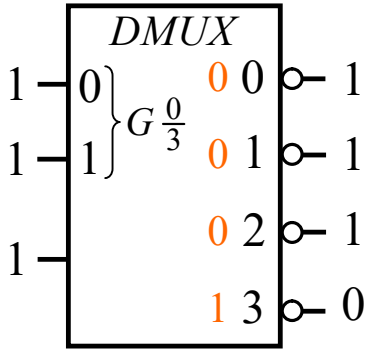
Ejemplos:

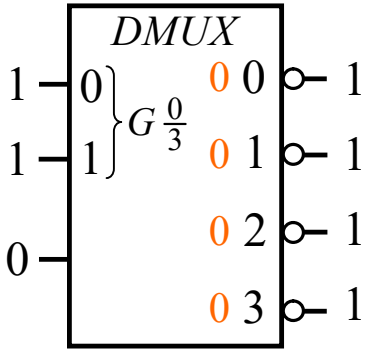

DMUX
1
0
0
0
1
G 0/3
0
1
2
3
0
0
0
0


DX
1
0
1
0
1
G 0/3
0
1
2
3
0
1
0
0

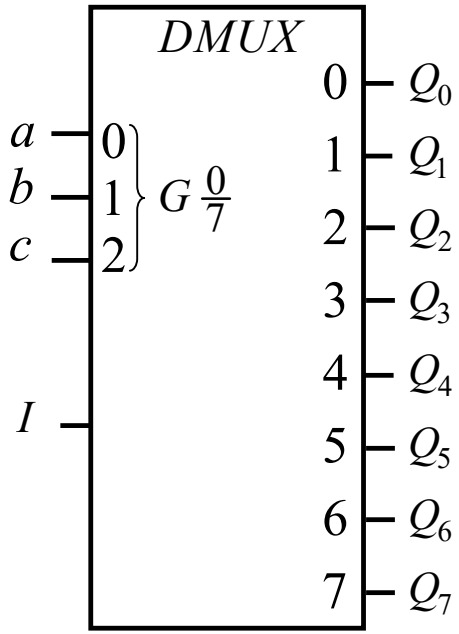

DMUX
0
0
0
0
1
G 0/3
0
1
2
3
0
0
0
0

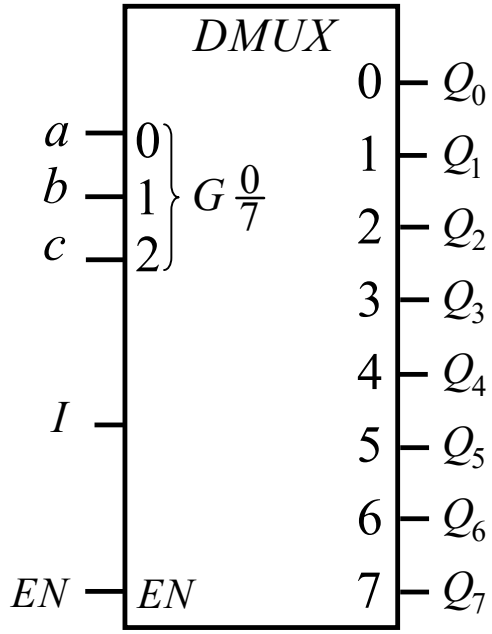

DX
0
1
1
0
1
G 0/3
0
1
2
3
0
0
1
0

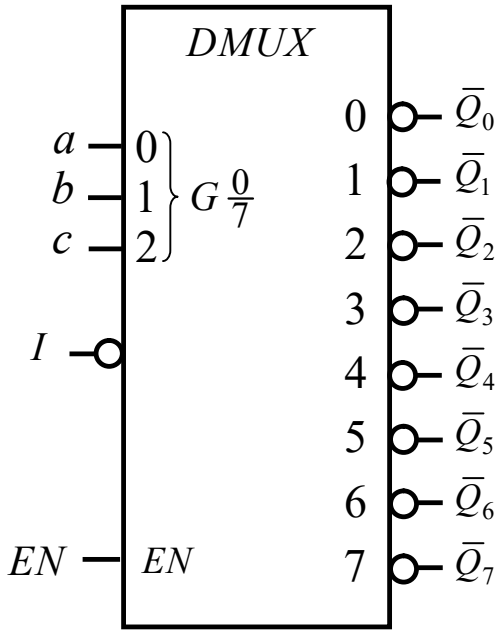

DMUX
1
1
1
0
1
G 0/3
0 0
0 1
0 2
1 3
1
1
1
0


DMUX
1
1
0
0
1
G 0/3
0 0
0 1
0 2
0 3
1
1
1
1

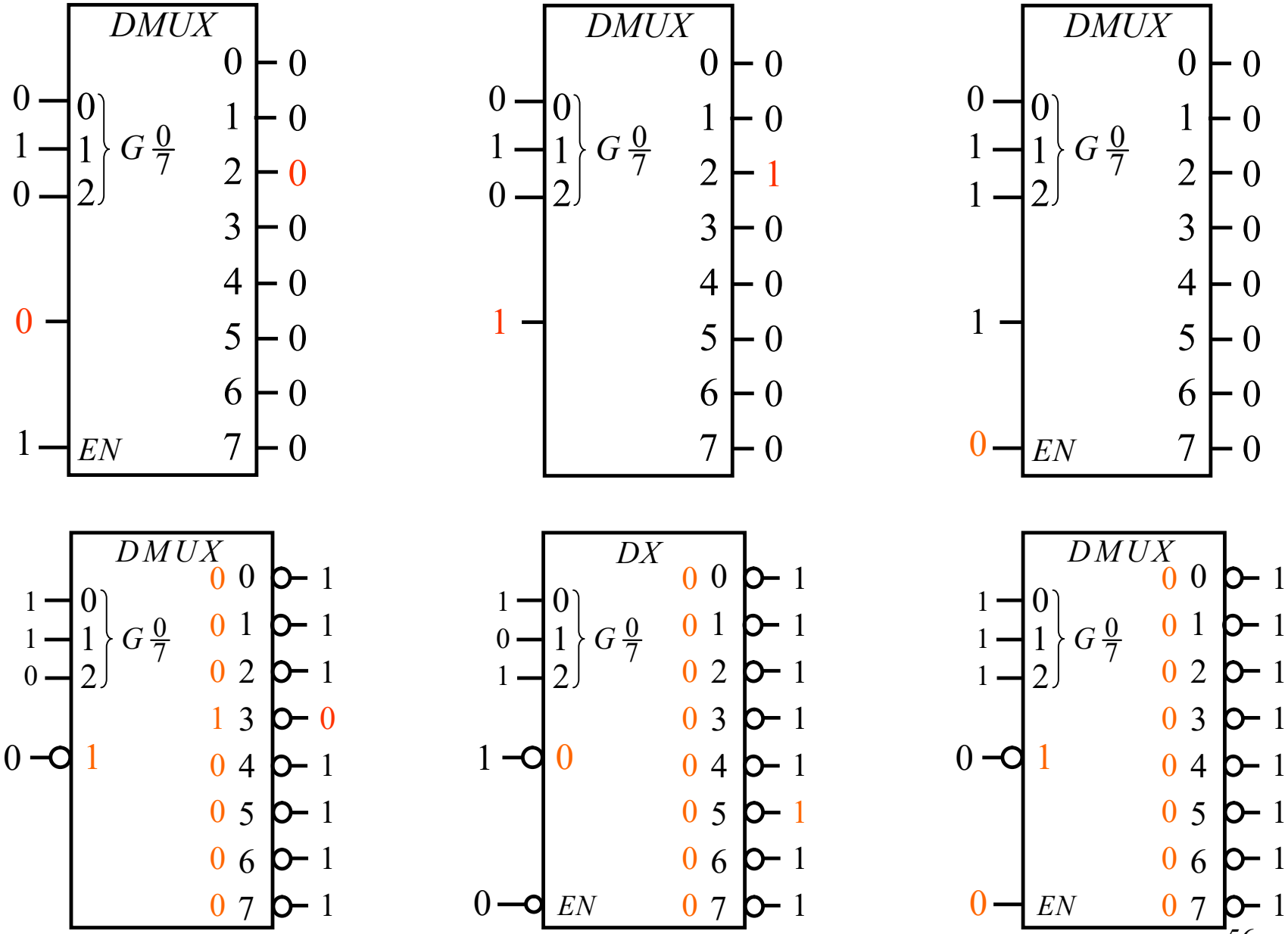
Demultiplexor de 8 canales (de salida)





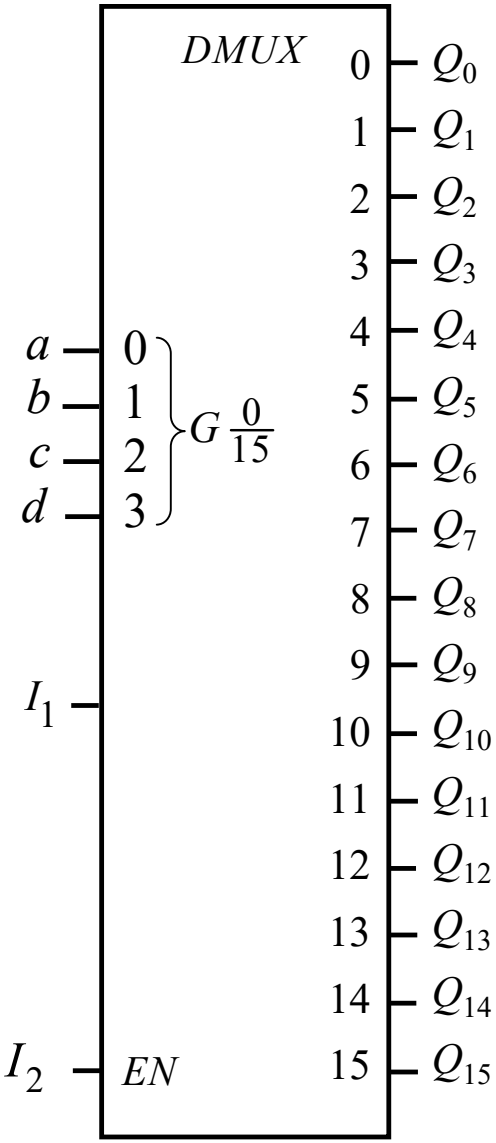


Ejemplos:

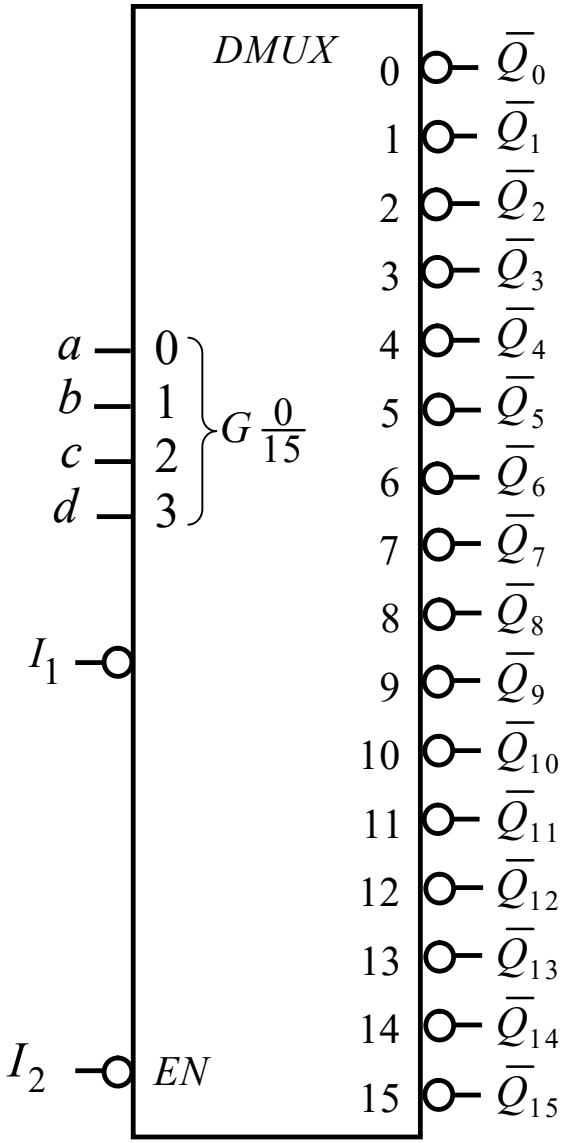


Demultiplexores de 16 canales (de salida)

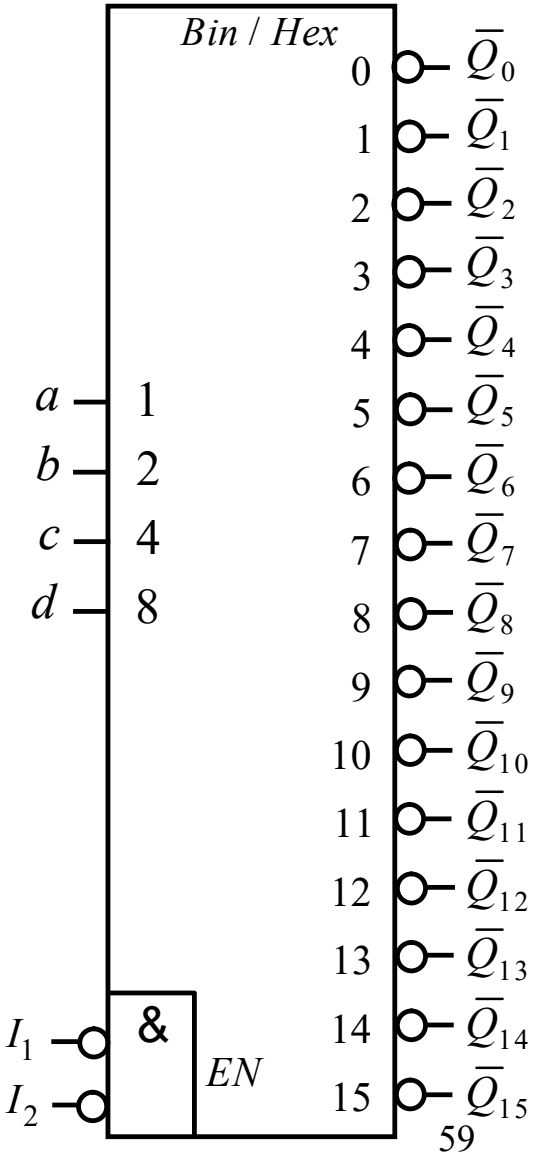
si



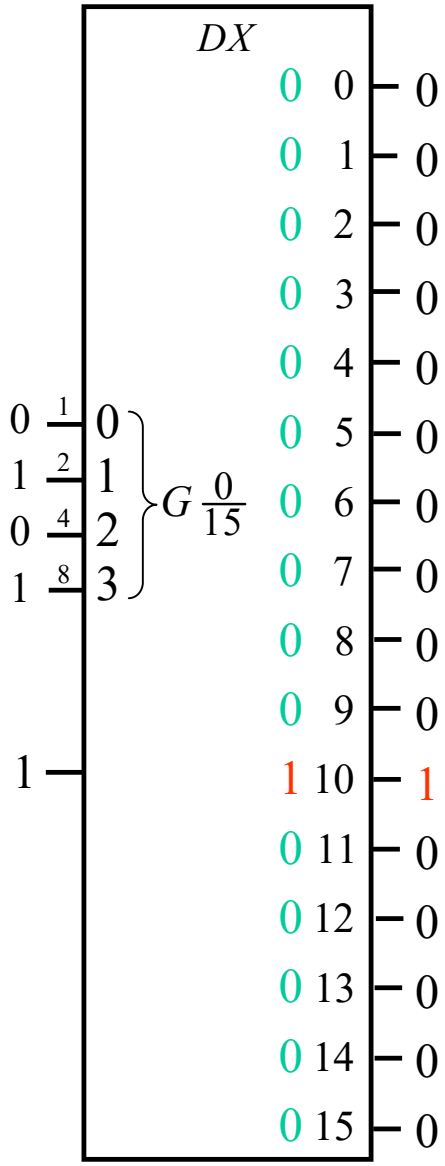
74*xxx*154 no

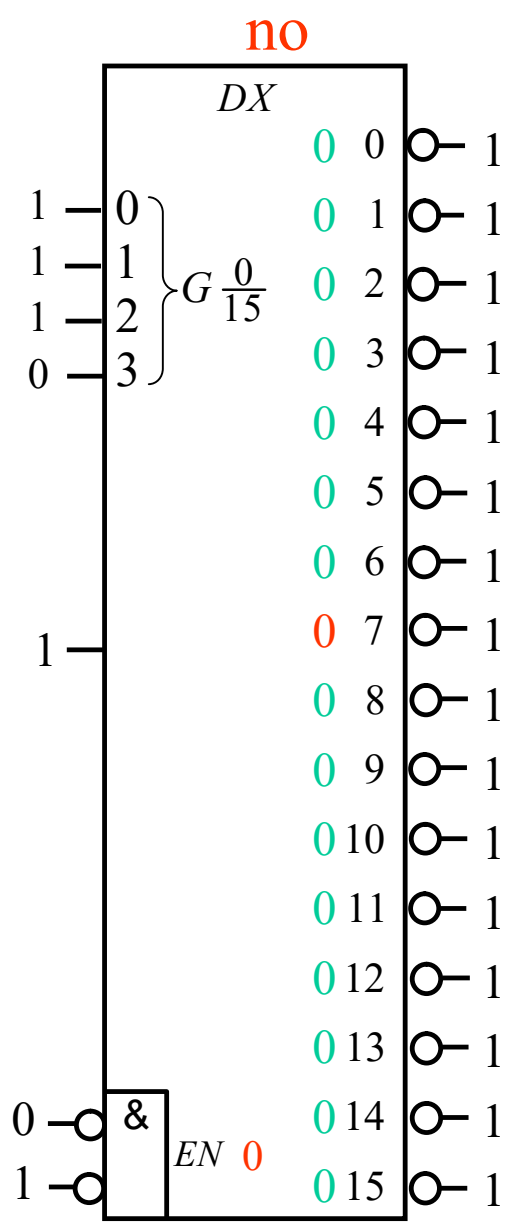


74*xxx*154 no



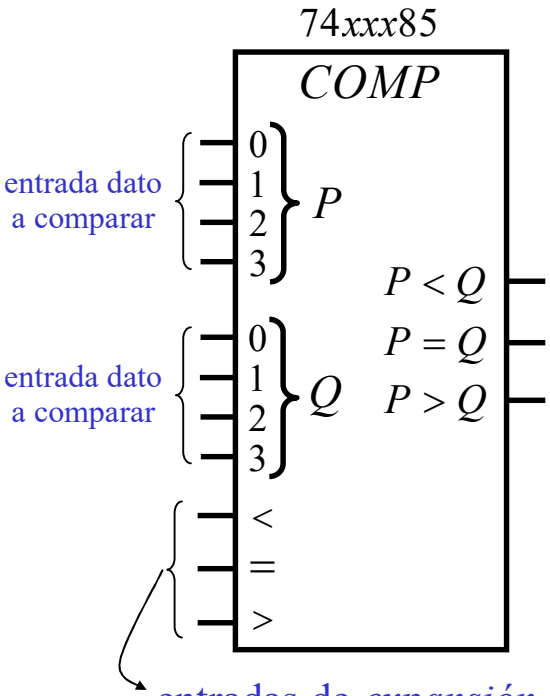
Ejemplos:





▪ Comparadores de magnitud de *n* bits

DEF.: Un comparador de magnitud es un circuito combinacional que indica si dos magnitudes de *n* bits, representadas en binario natural, son iguales o, en el caso de que no sean iguales, cual de ellas es la mayor.

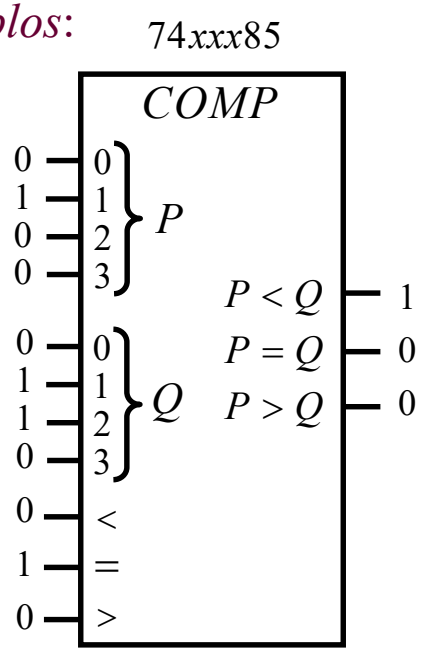


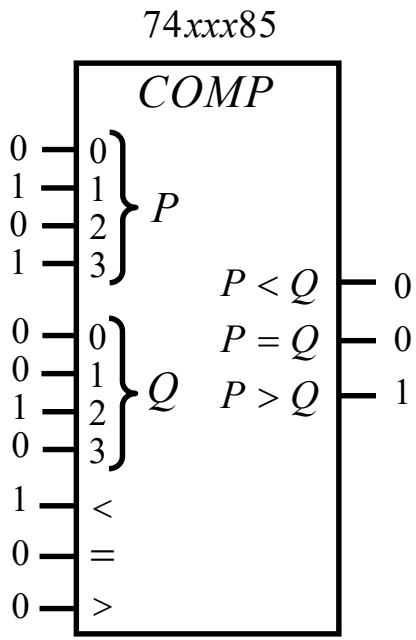
P Q	$<$	$=$	$>$	$P<Q$	$P=Q$	$P>Q$
$P<Q$	X	X	X	1	0	0
$P>Q$	X	X	X	0	0	1
$P=Q$	1	0	0	1	0	0
$P=Q$	0	1	0	0	1	0
$P=Q$	0	0	1	0	0	1

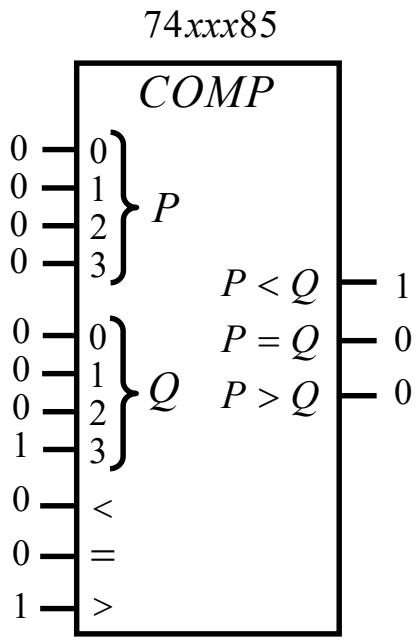
entradas de *expansión* o de *extensión*. Sirven para indicar qué salida (o salidas) se pone a 1 cuando las magnitudes (números) a comparar son iguales.

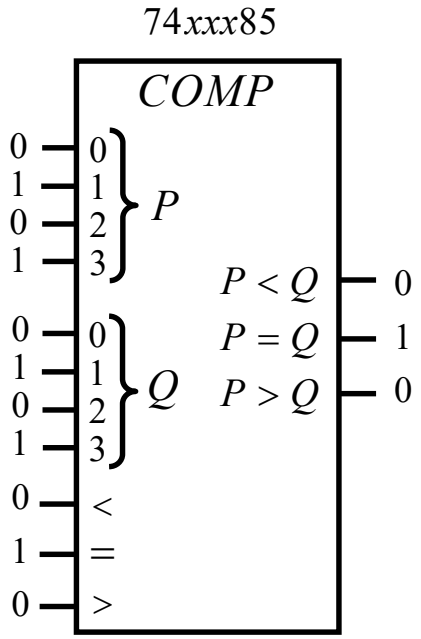
Nota: se comparan magnitudes (números binarios) <u>sin signo</u>

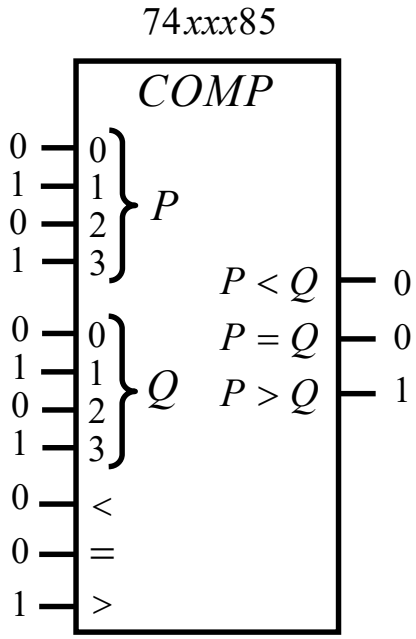
Ejemplos:

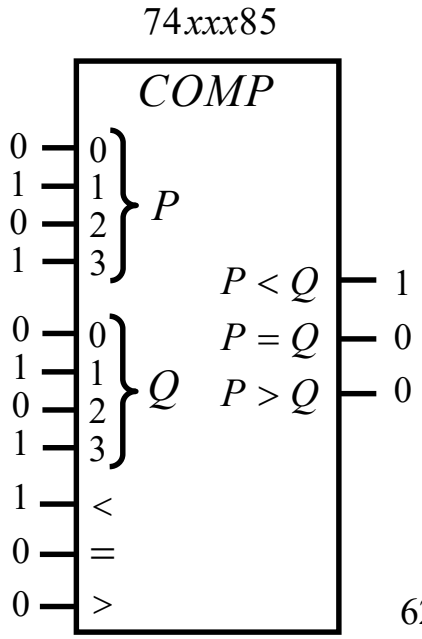

74xxx85
COMP
0 1 0 0
0 1 2 3
P
0 1 1 0
0 1 2 3
Q
0 1 0
<
=
>
P < Q
P = Q
P > Q
1 0 0


74xxx85
COMP
0 1 0 1
0 1 2 3
P
0 0 1 0
0 1 2 3
Q
1 0 0
<
=
>
P < Q
P = Q
P > Q
0 0 1


74xxx85
COMP
0 0 0 0
0 1 2 3
P
0 0 0 1
0 1 2 3
Q
0 0 1
<
=
>
P < Q
P = Q
P > Q
1 0 0


74xxx85
COMP
0 1 0 1
0 1 2 3
P
0 1 0 1
0 1 2 3
Q
0 1 0
<
=
>
P < Q
P = Q
P > Q
0 1 0


74xxx85
COMP
0 1 0 1
0 1 2 3
P
0 1 0 1
0 1 2 3
Q
0 0 1
<
=
>
P < Q
P = Q
P > Q
0 0 1


74xxx85
COMP
0 1 0 1
0 1 2 3
P
0 1 0 1
0 1 2 3
Q
1 0 0
<
=
>
P < Q
P = Q
P > Q
1 0 0

Ejercicio 1: Dados 2 números binarios A y B de 4 bits (sin signo), diseña un circuito que proporcione el mayor de ambos o bien el valor cero en el caso de que sean iguales. Para resolver este ejercicio no se pueden utilizar puertas lógicas.

Pregunta: ¿Cuántas entradas y cuántas salidas tiene el circuito a diseñar?

Ejercicio 2: Diseña un circuito al que llegan 3 números binarios A, B y C de 4 bits y una señal de control d. El circuito funciona de la siguiente manera:

_ Si $d = 1$, entonces el circuito debe indicar la relación entre B y C ($B > C$, $B < C$ o $B = C$)

_ Si $d = 0$, entonces el circuito debe indicar la relación entre B y A ($B > A$, $B < A$ o $B = A$)

Para resolver este ejercicio no se pueden utilizar puertas lógicas

Pregunta: ¿Cuántas entradas y cuántas salidas tiene el circuito a diseñar?

- Detectores / Generadores de paridad

DEF.: Un detector de paridad es un circuito combinacional que detecta (indica) si en un grupo de *n* bits el número de bits que están a 1 es un número par (≡ *detector de paridad par*) o impar (≡ *detector de paridad impar*).

Se cumple que:

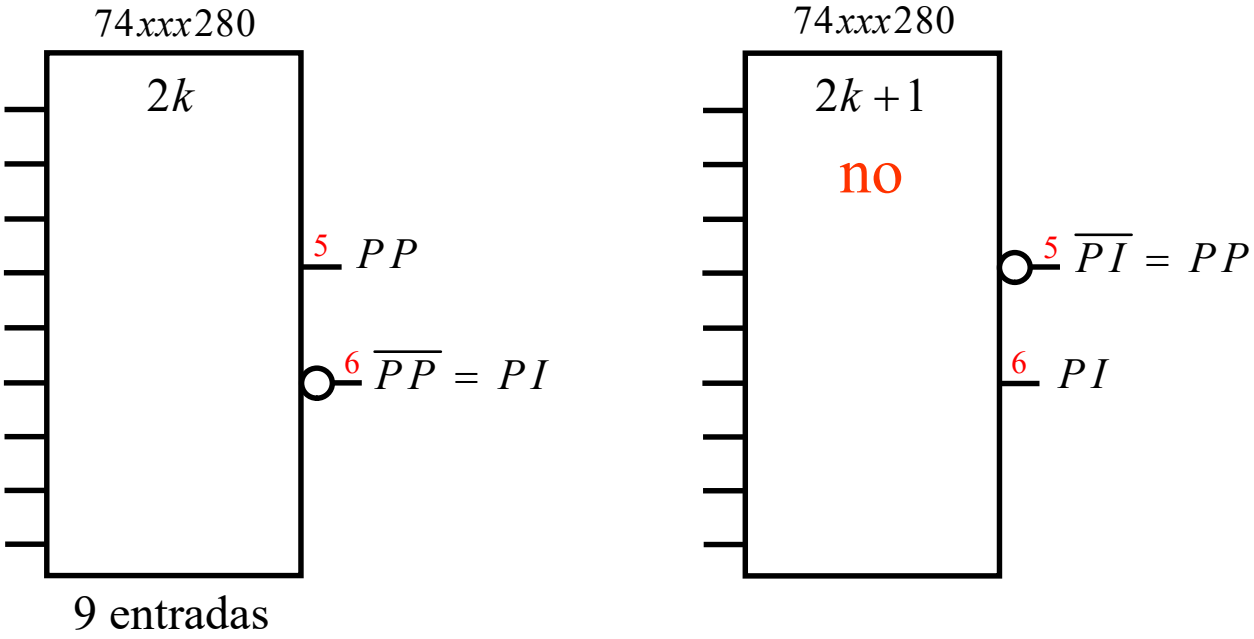
$$PP(a_8\, a_7\, a_6\, a_5\, a_4\, a_3\, a_2\, a_1\, a_0) = \overline{a_8 \oplus a_7 \oplus a_6 \oplus a_5 \oplus a_4 \oplus a_3 \oplus a_2 \oplus a_1 \oplus a_0}$$

$$PI(a_8\, a_7\, a_6\, a_5\, a_4\, a_3\, a_2\, a_1\, a_0) = \overline{PP} = a_8 \oplus a_7 \oplus a_6 \oplus a_5 \oplus a_4 \oplus a_3 \oplus a_2 \oplus a_1 \oplus a_0$$

Nota:

$$x_1 \oplus x_2 \oplus x_3 \cdots \oplus x_n = \begin{cases} 1 & si\ el\ número\ de\ variables\ que\ están\ a\ 1\ es\ un\ número\ impar \\ 0 & si\ es\ par \end{cases}$$

$$\overline{x_1 \oplus x_2 \oplus x_3 \cdots \oplus x_n} = \begin{cases} 1 & si\ el\ número\ de\ variables\ que\ están\ a\ 1\ es\ un\ número\ par \\ 0 & si\ es\ impar \end{cases}$$

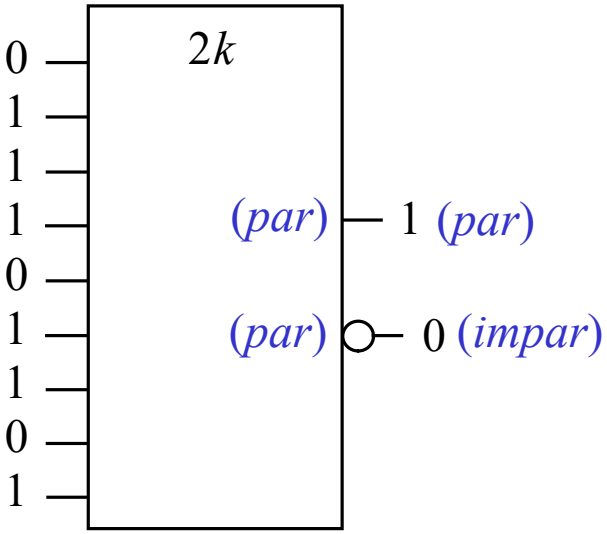


Nota 1: $2k$ con $k \in \mathbb{N}$ siempre es un número *par* y por lo tanto $2k$ se utiliza para indicar un *detector de paridad par* (el 0 se considera un número *par*).

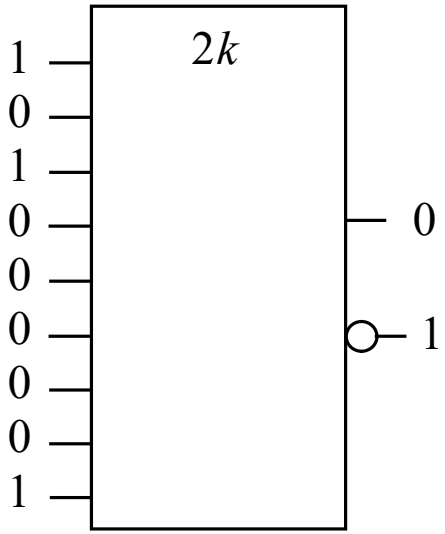
Nota 2: $2k + 1$ con $k \in \mathbb{N}$ siempre es un número *impar* y por lo tanto $2k + 1$ se utiliza para indicar un *detector de paridad impar*.

Nota 3: *en los circuitos anteriores los estados lógicos* <u>*internos*</u> *de las salidas siempre valen lo mismo, sin embargo los estados lógicos* <u>*externos*</u> *de las salidas son siempre distintos* (*uno es el negado del otro*). Esto hace que cuando una salida indica *paridad par* la otra indica *paridad impar*

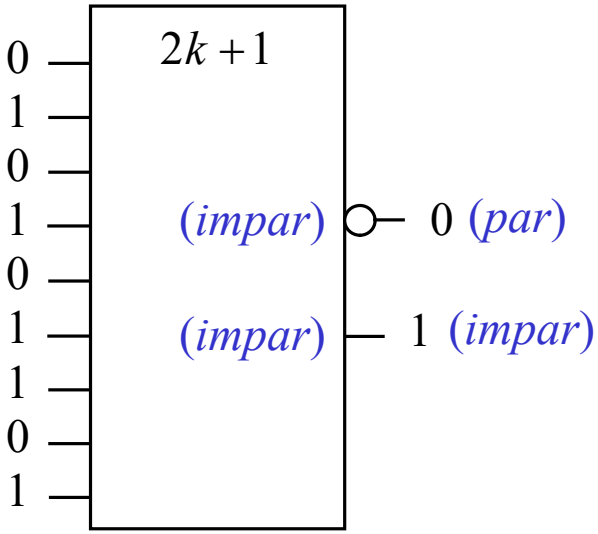
Ejemplos:



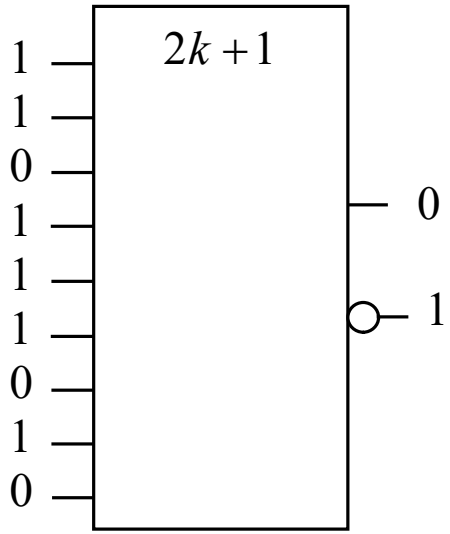
6



3

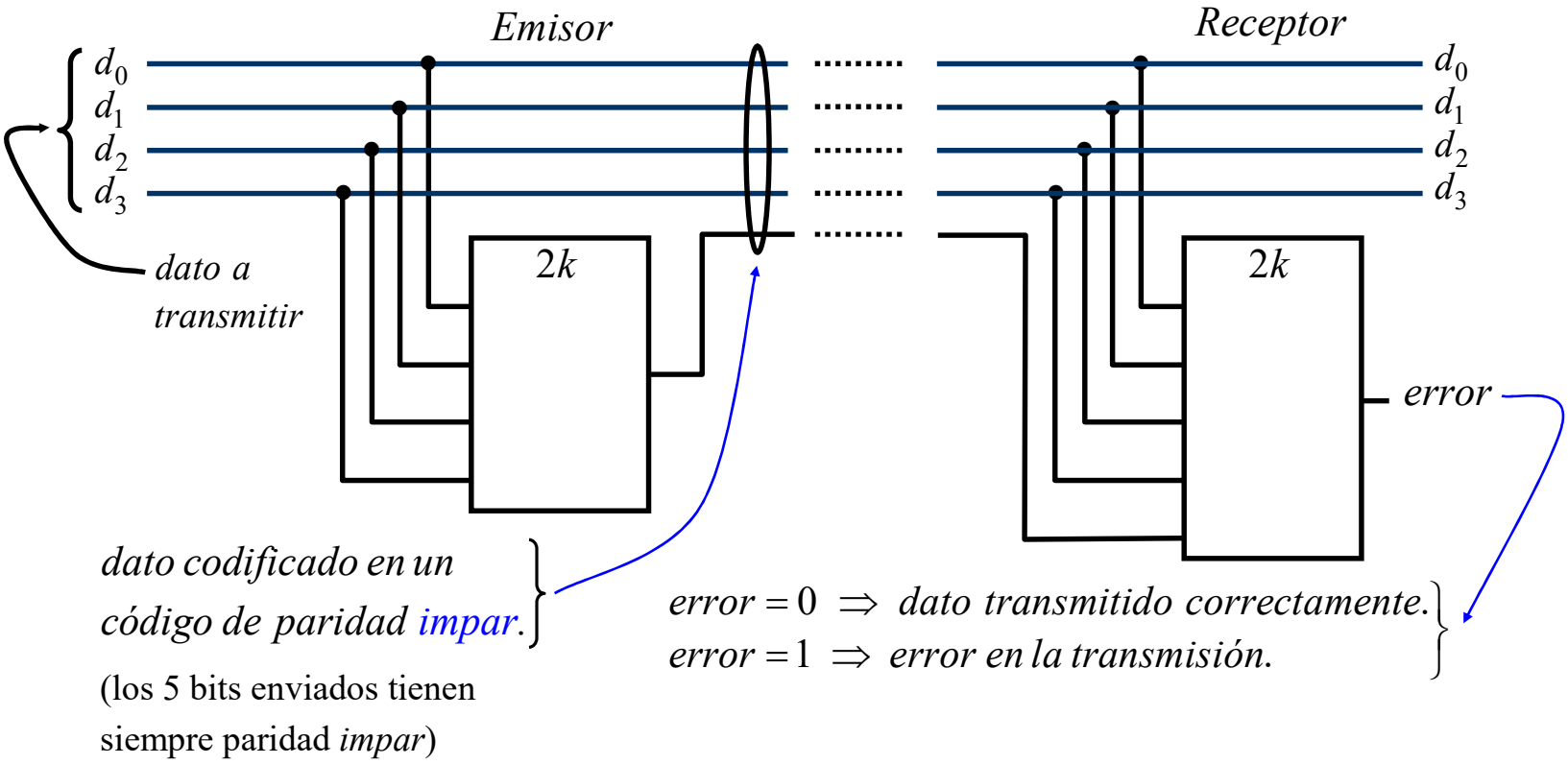


5



6

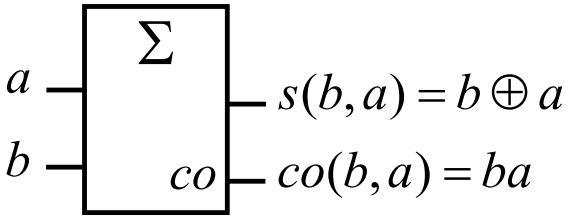
Ejemplo de aplicación:



Curiosidad: *los microprocesadores Pentium comprobaban la paridad de la información enviada tanto por el bus de datos como por el bus de direcciones.*

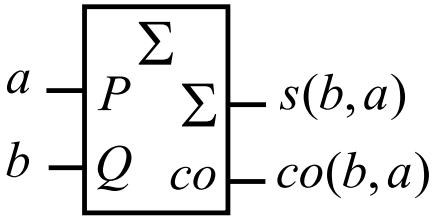
▪ Circuitos sumadores: son circuitos combinacionales que realizan sumas *aritméticas*

_ Semisumador de 1 bit: *si los valores lógicos* (1_s *y* 0_s) *presentes en los terminales de entrada y de salida del circuito indicado a continuación se interpretan como* 1_s *y* 0_s *binarios* ($1_2 \equiv 1_{lógico}$ y $0_2 \equiv 0_{lógico}$), *entonces se puede decir que el circuito indicado a continuación realiza la suma aritmética de dos dígitos binarios* (*b y a*) *indicada en la parte derecha.*



$$\begin{array}{r} b \\ +\ a \\ \hline co\ \ s \end{array}$$

|||



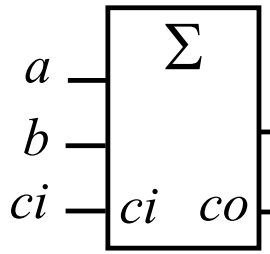
$$\begin{array}{r} b \\ +\ a \\ \hline co\ \ s \end{array} \equiv \begin{array}{r} 0 \\ +\ 1 \\ \hline 0\ \ 1 \end{array}$$

b	a	co (2)	s (1)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Nota: se aplica la siguiente analogía: $1_2 \equiv 1_{lógico}$ y $0_2 \equiv 0_{lógico}$

_ Sumador total (o completo) de 1 bit (*full-adder*): el circuito indicado a continuación realizan la *suma aritmética* de tres dígitos binarios (ci, b, a)

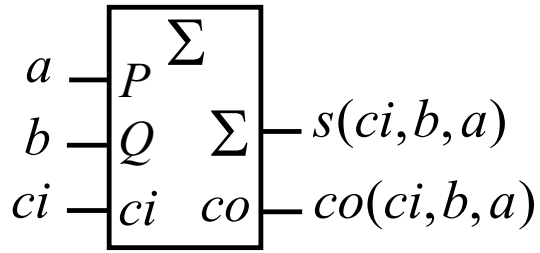
$$\begin{array}{r} ci \\ b \\ +\ a \\ \hline co\ \ s \end{array}$$



$$s(ci,b,a) = ci \oplus b \oplus a$$

$$co(ci,b,a) = ci\left(b+a\right) + ba = ci\left(b \oplus a\right) + ba$$

|||

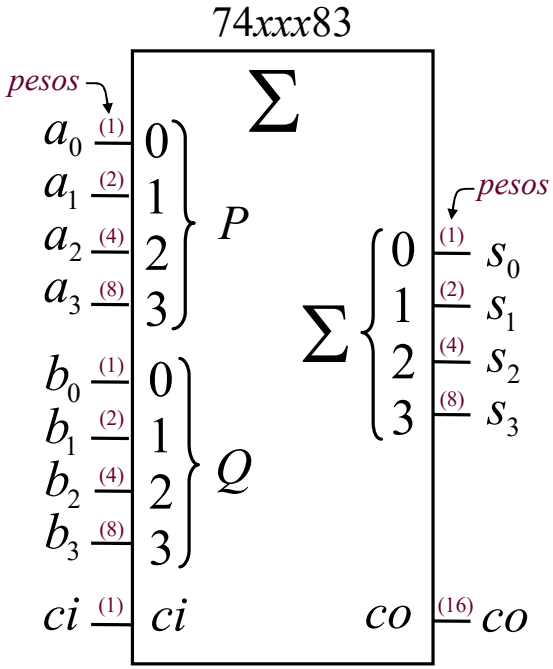


$$\begin{array}{r} {}^{2}\ {}^{1} \\ ci \\ b \\ +\ a \\ \hline co\ \ s \end{array} \equiv \begin{array}{r} 1 \\ 0 \\ +\ 1 \\ \hline 1\ \ 0 \end{array}$$

ci	b	a	co (2)	s (1)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Nota: se aplica la siguiente analogía: $1_2 \equiv 1_{lógico}$ y $0_2 \equiv 0_{lógico}$

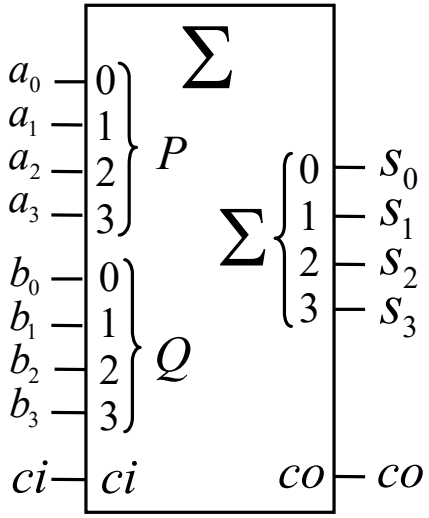
_ Sumador total de 4 bits: *si los valores lógicos* (1_s *y* 0_s) *presentes en los terminales de entrada y de salida del circuito indicado a continuación se interpretan como* 1_s *y* 0_s *binarios* ($1_2 \equiv 1_{lógico}$ y $0_2 \equiv 0_{lógico}$), *entonces se puede decir que el circuito indicado a continuación realiza la suma aritmética indicada en la parte derecha*



$$
\begin{array}{rccccc}
 & & & & & ci \\
 & & b_3 & b_2 & b_1 & b_0 \\
+ & & a_3 & a_2 & a_1 & a_0 \\
\hline
 & (16) & (8) & (4) & (2) & (1) \\
 & co & s_3 & s_2 & s_1 & s_0
\end{array}
$$

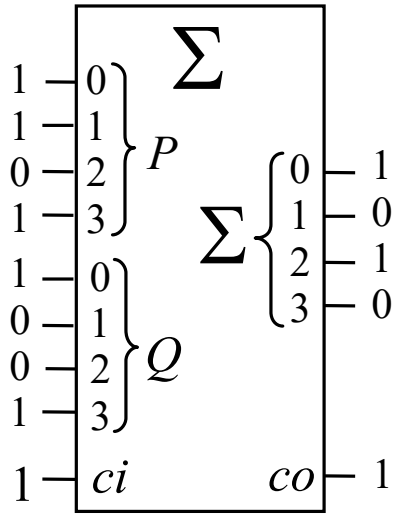
Nota: los nombres asignados a los terminales de *entrada* y de *salida* no forman parte del símbolo (sólo se han puesto para poder explicar fácilmente la operación aritmética que realiza el circuito al interpretar los 1_s y los 0_s lógicos como 1_s y 0_s binarios respectivamente)

Ejemplo:



$\Longleftrightarrow$

$$
\begin{array}{rrrrr}
 & & & & ci \\
 & b_3 & b_2 & b_1 & b_0 \\
+ & a_3 & a_2 & a_1 & a_0 \\
\hline
co & s_3 & s_2 & s_1 & s_0
\end{array}
$$

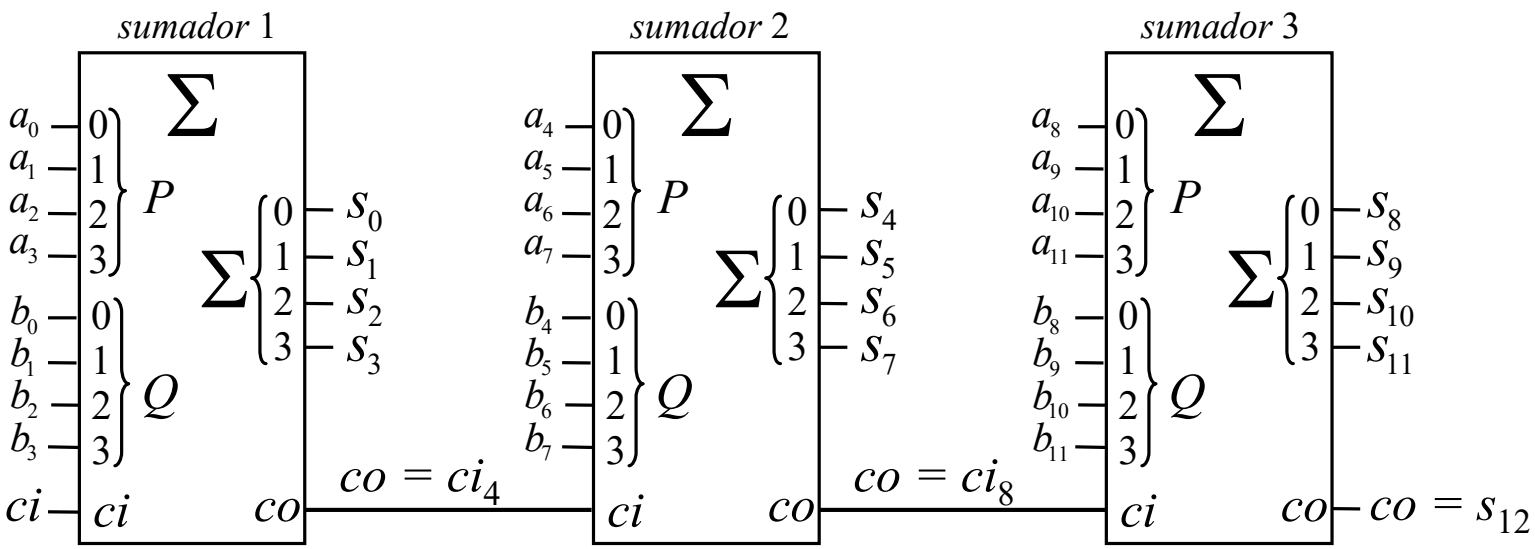


$\Longleftrightarrow$

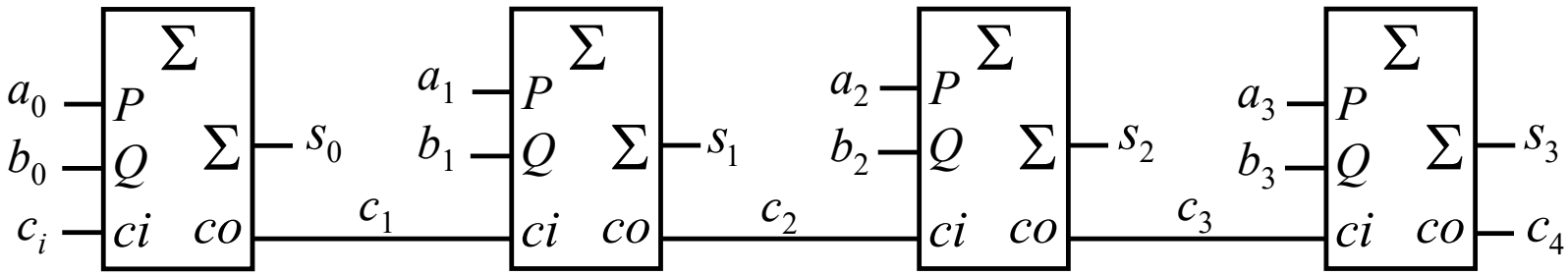
$$
\begin{array}{rrrrr}
{\scriptstyle 1} & & {\scriptstyle 1} & {\scriptstyle 1} & 1 \\
 & 1 & 0 & 1 & 1 \\
+ & 1 & 0 & 0 & 1 \\
\hline
 & 1 & 0 & 1 & 0 \quad 1
\end{array}
$$

Ejemplo de utilidad de la entrada *ci* (*carry input*): se efectúa una suma por partes (el *co* ≡ *carry output* de una suma pasa a ser el *ci* ≡ *carry input* de la siguiente suma)

$$
\begin{array}{rrrrr|rrrr|rrrr}
 & & & & ci_8 & & & & ci_4 & & & & ci \\
 & a_{11} & a_{10} & a_9 & a_8 & a_7 & a_6 & a_5 & a_4 & a_3 & a_2 & a_1 & a_0 \\
+ & b_{11} & b_{10} & b_9 & b_8 & b_7 & b_6 & b_5 & b_4 & b_3 & b_2 & b_1 & b_0 \\
\hline
 & \underbrace{s_{12} \; s_{11} \; s_{10} \; s_9 \; s_8}_{sumador\ 3} & & & & \underbrace{s_7 \; s_6 \; s_5 \; s_4}_{sumador\ 2} & & & & \underbrace{s_3 \; s_2 \; s_1 \; s_0}_{sumador\ 1} & & &
\end{array}
$$



Ejemplo de implementación de un sumador total de 4 bits con sumadores totales de 1 bit:



$$s_0 = ci \oplus b_0 \oplus a_0$$

$$c_1 = c_0 b_0 + c_0 a_0 + b_0 a_0$$

$$s_1 = c_1 \oplus b_1 \oplus a_1$$

$$c_2 = c_1 b_1 + c_1 a_1 + b_1 a_1$$

$$s_2 = c_2 \oplus b_2 \oplus a_2$$

$$c_3 = c_2 \cdot b_2 + c_2 \cdot a_2 + b_2 a_2$$

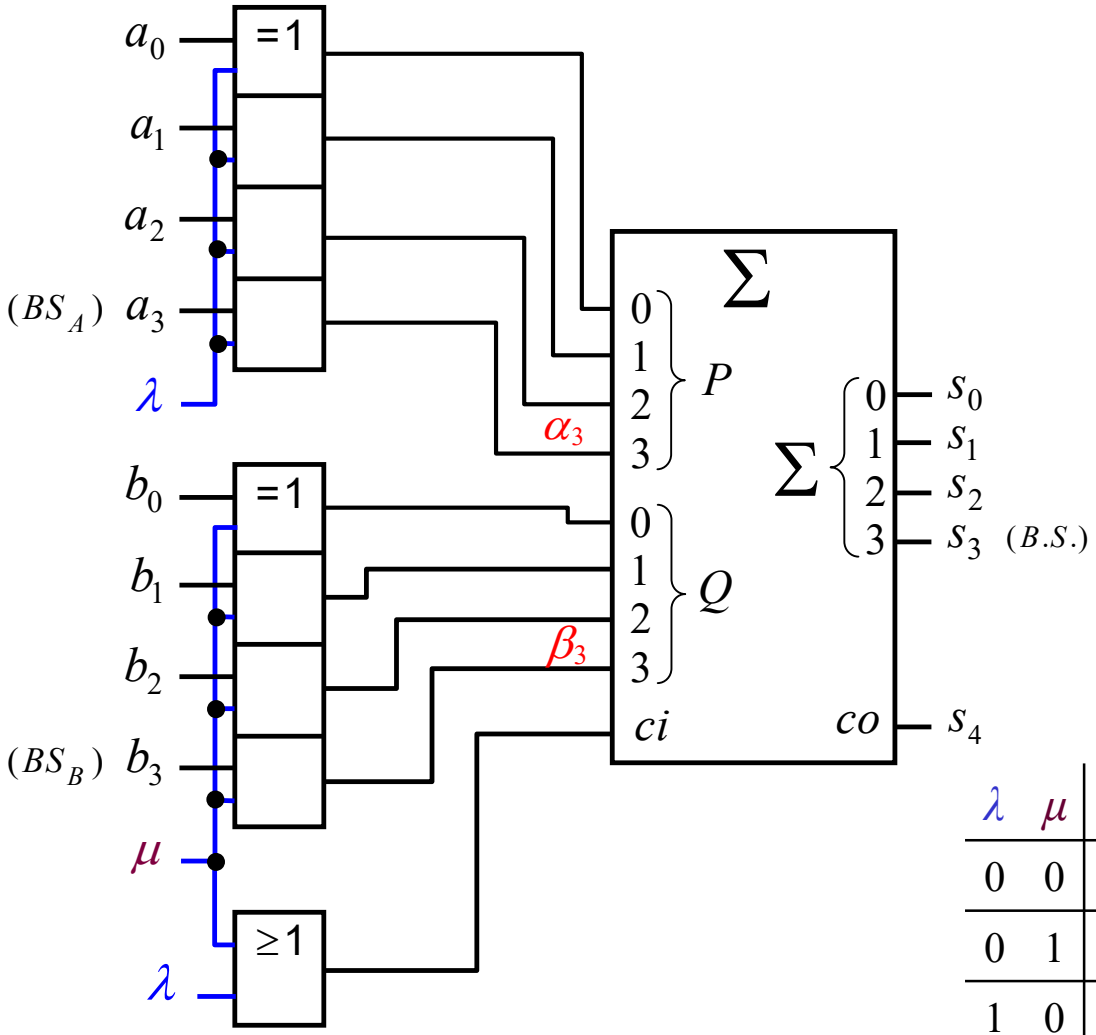
$$s_3 = c_3 \oplus b_3 \oplus a_3$$

$$c_4 = co = c_3 b_3 + c_3 a_3 + b_3 a_3$$

$$\begin{array}{rccccc} & & & & & ci \\ & & b_3 & b_2 & b_1 & b_0 \\ + & & a_3 & a_2 & a_1 & a_0 \\ \hline & c_4 & s_3 & s_2 & s_1 & s_0 \end{array}$$

Pasatiempo: intenta diseñar un sumador total de 1 *bit a partir de* <u>*semisumadores*</u> *de* 1 *bit y del menor número posible de puertas lógicas básicas.*

Circuito sumador/restador de números de 4 bits codificados en el *cca*2.



$$A(\overset{BS}{a_3}\, a_2 a_1 a_0)_{cca2}$$

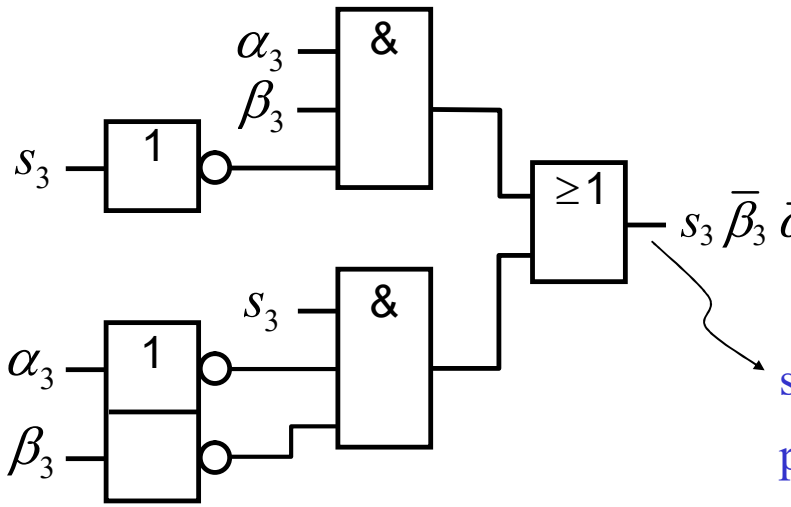
$$B(\overset{BS}{b_3}\, b_2 b_1 b_0)_{cca2}$$

$$S(\overset{BS}{s_3}\, s_2 s_1 s_0)_{cca2}$$

λ	μ	*Operación realizada*
0	0	$A+B$
0	1	$A-B \rightarrow A + ca2(B) = A + ca1(B) + 1$
1	0	$-A+B \rightarrow ca2(A)+B = ca1(A) + B + 1$
1	1	*prohibida*

Nota: μ y λ son entradas auxiliares que permiten especificar la operación a realizar

Circuito detector de desbordamiento (*overflow*).



$$s_3\,\overline{\beta}_3\,\overline{\alpha}_3 + \overline{s}_3\,\beta_3\,\alpha_3 \equiv desbordamiento\ (overflow)$$

si en esta salida hay un 1 significa que se ha producido un desbordamiento.

Recordatorio: ocurre un desbordamiento cuando al sumar dos números *positivos* se obtiene como resultado un número *negativo*. También ocurre un desbordamiento cuando al sumar dos números *negativos* se obtiene como resultado un número *positivo*.

Aplicaciones de los circuitos sumadores:

_ Sumar números binarios

_ Restar/sumar números representados en el *cca*2.

_ Diseño de convertidores de código.

_ Implementación de comparadores de magnitud.

_ Implementación de detectores/generadores de paridad.

_ Implementación de algunas funciones lógicas.

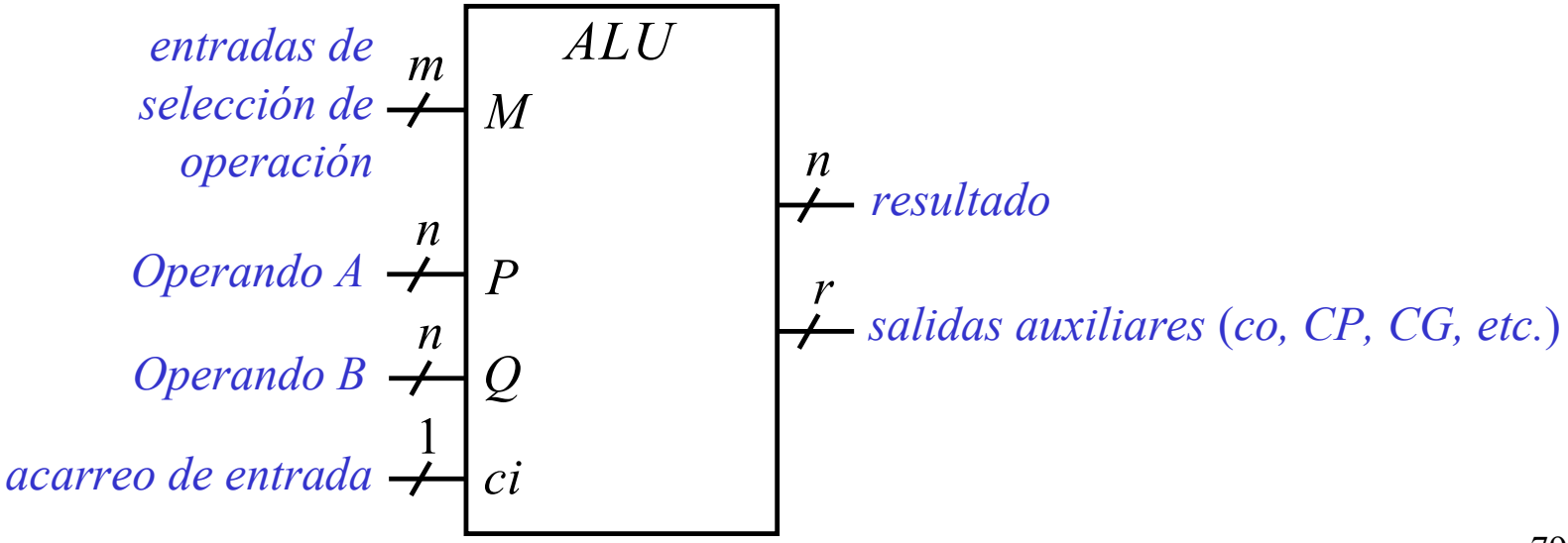
_ etc.

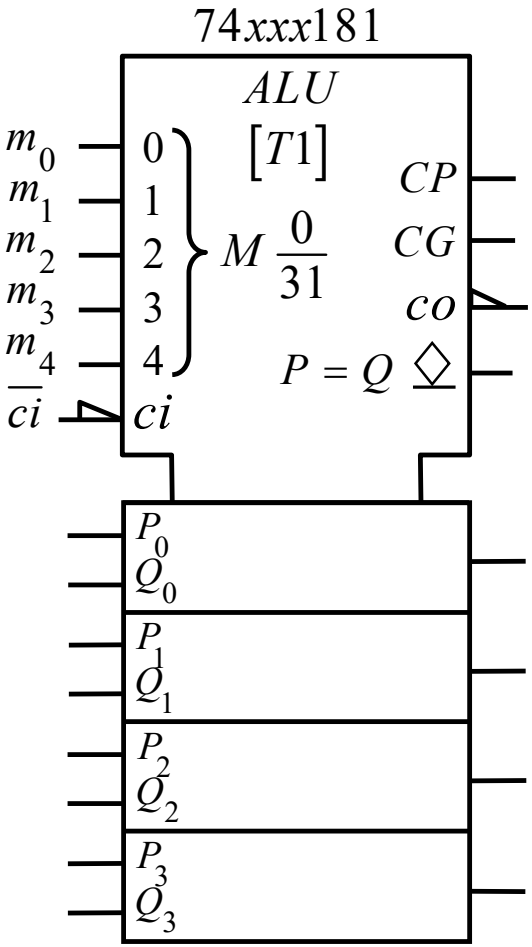
▪ Unidad aritmético-lógica (ALU: *Arithmetic logic unit*)

Es un circuito *combinacional* que puede realizar tanto operaciones *aritméticas* como *lógicas*. Las ALUs tienen dos tipos de entradas:

▪ *Entradas de selección*: sirven para indicarle a la *ALU* la operación (*aritmética* o *lógica*) a realizar. El repertorio de instrucciones que puede realizar una ALU se indica en una tabla (ver siguiente diapositiva)

▪ *Entradas de datos*: sirven para proporcionarle a la *ALU* los datos con los que debe operar (hay operaciones en las que sólo se utiliza un operando).





$$operandos \begin{cases} P : P_3P_2P_1P_0 \\ Q : Q_3Q_2Q_1Q_0 \end{cases}$$

m_3	m_2	m_1	m_0	*Logic* $m_4 = 1$	*Arithmetic* $m_4 = 0$
0	0	0	0	$\overline{P}$	P *plus* ci
0	0	0	1	$\overline{P+Q}$	$P+Q$ *plus* ci
0	0	1	0	$\overline{P}Q$	$P+\overline{Q}$ *plus* ci
0	0	1	1	0000	$(-1)_{cca2}$ *plus* ci
0	1	0	0	$\overline{PQ}$	P *plus* $(P\cdot\overline{Q})$ *plus* ci
0	1	0	1	$\overline{Q}$	$P\cdot\overline{Q}$ *plus* $(P+Q)$ *plus* ci
0	1	1	0	$P\oplus Q$	$(-1)_{cca2}$ *plus* P *minus* Q *plus* ci
0	1	1	1	$P\cdot\overline{Q}$	$(-1)_{cca2}$ *plus* $P\cdot\overline{Q}$ *plus* ci
1	0	0	0	$\overline{P}+Q$	P *plus* $P\cdot Q$ *plus* ci
1	0	0	1	$\overline{P\oplus Q}$	P *plus* Q *plus* ci
1	0	1	0	Q	$P\cdot Q$ *plus* $(P+\overline{Q})$ *plus* ci
1	0	1	1	$P\cdot Q$	$(-1)_{cca2}$ *plus* $P\cdot Q$ *plus* ci
1	1	0	0	1111	P *plus* P *plus* ci
1	1	0	1	$P+\overline{Q}$	P *plus* $(P+Q)$ *plus* ci
1	1	1	0	$P+Q$	P *plus* $(P+\overline{Q})$ *plus* ci
1	1	1	1	P	$(-1)_{cca2}$ *plus* P *plus* ci

$+ \equiv$ *suma lógica bit a bit* (*bitwise*) *plus* / *minus* $\equiv$ *suma* / *resta aritmética*

En las operaciones aritméticas, *minus* Q representa la operación $ca2(Q)$

Ejemplo:

m_4	m_3	m_2	m_1	m_0	*Operación realizada*
0	0	0	0	0	$P\ plus\ ci$
1	0	0	0	0	$\overline{P}\ \equiv ca1(P)$

$P: P_3P_2P_1P_0$

(operación *aritmética*: $m_4 = 0$)

$$\left.\begin{array}{r} P: 1101 \\ ci: 1 \end{array}\right\} \Rightarrow \quad P\ plus\ ci:\ 1110 \quad \equiv 13 + 1 = 14$$

(operación *lógica*: $m_4 = 1$)

$$P: 1101 \quad \Rightarrow \quad \overline{P}: 0010 \quad \equiv \text{operación bit a bit (}bitwise\text{)}$$

Notas:

- $+ \equiv$ *suma lógica bit a bit* (*en estas sumas no se producen acarreos y* 1+1 = 1)
- *plus*/*minus* $\equiv$ *suma*/*resta aritmética* (*aquí sí se pueden producir acarreos*)
- las operaciones lógicas se realizan '*bit a bit*' (*bit-wise*)

Ejercicio 3: Diseñar un circuito que calcule la suma de 3 números binarios $A = 1011_2$, $B = 010_2$ y $C = 100_2$. Nota: no se pueden utilizar puertas lógicas.

Ejercicio 4: Diseñar un convertidor de palabras código que en función del valor de una entrada auxiliar α realice las siguientes conversiones:

_ para $\alpha = 0$ se debe realizar la conversión de $BCD_{natural} \rightarrow BCD_{exceso\ 3}$

_ para $\alpha = 1$ se debe realizar la conversión de $BCD_{exceso\ 3} \rightarrow BCD_{natural}$

Nota: sólo se pueden utilizar bloques funcionales combinacionales y 1 puerta lógica!!!

Ejercicio 5: Diseñar un convertidor de palabras código que en función del valor de una entrada auxiliar α realice las siguientes conversiones:

_ para $\alpha = 0$ se debe realizar la conversión de $BCD_{exceso\ 3} \rightarrow BCD_{Aiken}$

_ para $\alpha = 1$ se debe realizar la conversión de $BCD_{Aiken} \rightarrow BCD_{exceso\ 3}$

Nota: sólo se pueden utilizar bloques funcionales combinacionales y el menor número posible de puertas XOR.

• Prácticas: (se realizan en el laboratorio de Electrónica, en la 3º planta)

· El enunciado de las prácticas se publicará en https://moovi.uvigo.gal

· Las fechas en las que se realizarán las prácticas en el laboratorio de Electrónica son:

Práctica 1: grupos 2, 5, 6 y 8 (lunes): 2 de diciembre

grupos 4 y 7 (jueves): 28 de noviembre

grupos 1 y 3 (viernes): 29 de noviembre

Práctica 2: grupos 2, 5, 6 y 8: 9 de diciembre

grupos 4 y 7: 12 de diciembre

grupos 1 y 3: 13 de diciembre

Práctica 3: grupos 2, 5, 6 y 8: 16 de diciembre

grupos 4 y 7: 19 de diciembre

grupos 1 y 3: 20 de diciembre

Muy importante: al laboratorio hay que asistir con los problemas de diseño que se plantean en los enunciados de las prácticas resueltos en hojas de papel.

Tema 5